# iDataType_使用说明

## 1. 工具概述

**iDataType**为产品经理、系统工程师、软件架构师等设计人员提供了一套的图形化界面，以便于他们进行产品设计和开发，工具支持将现有的代码文件快速导入到工具中，以便于进一步配置和修改。设计人员可以通过图形界面手动创建和配置数据类型，如类、结构体、枚举等。工具可以根据配置的类型动态生成多种编程语言的源代码文件。

## 2. 下载启动

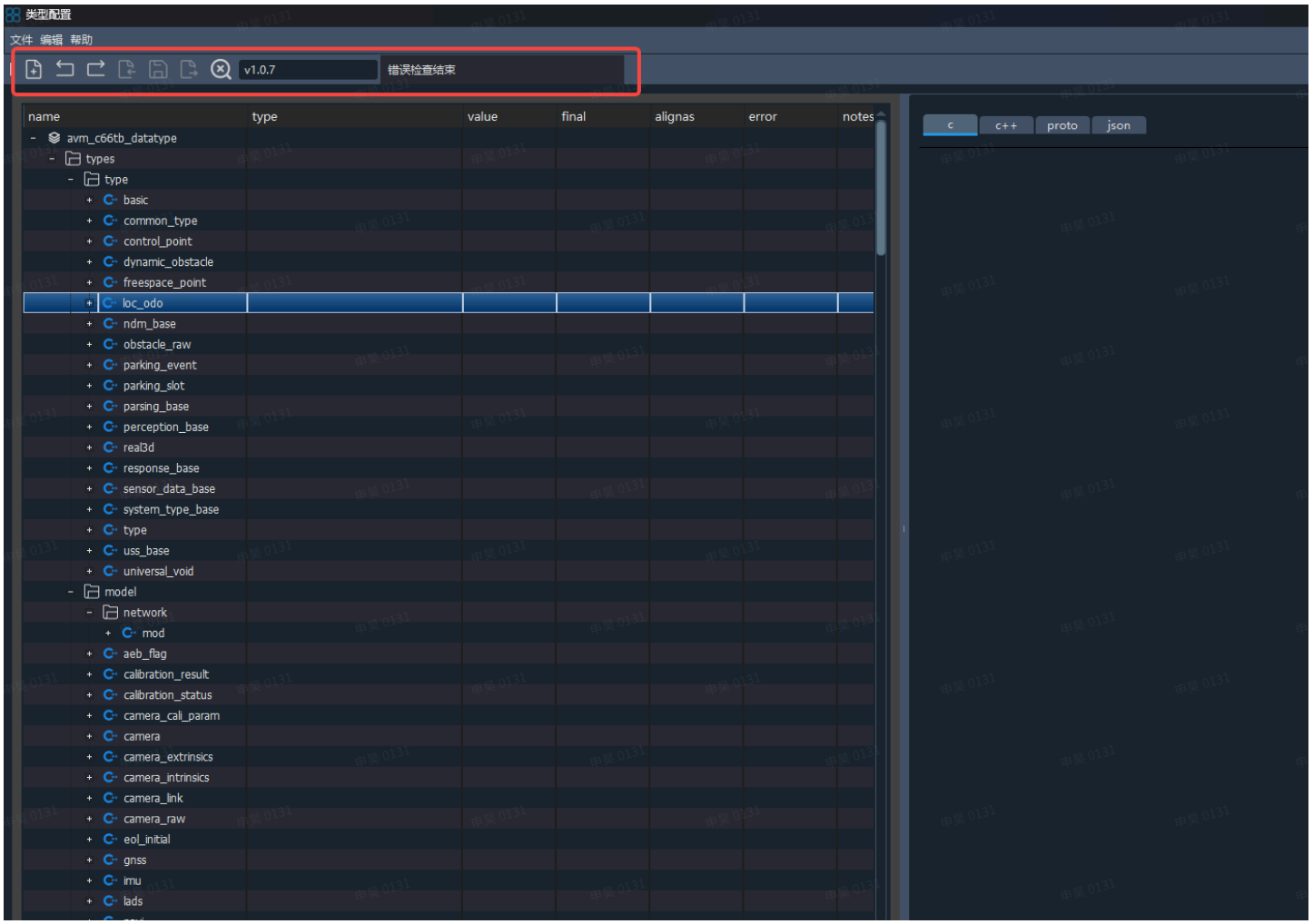
访问网站（类型配置工具_发布记录），选择版本下载对应安装包。

## 3. 界面介绍

启动类型配置工具后，进入主界面。主界面包括以下几个部分：

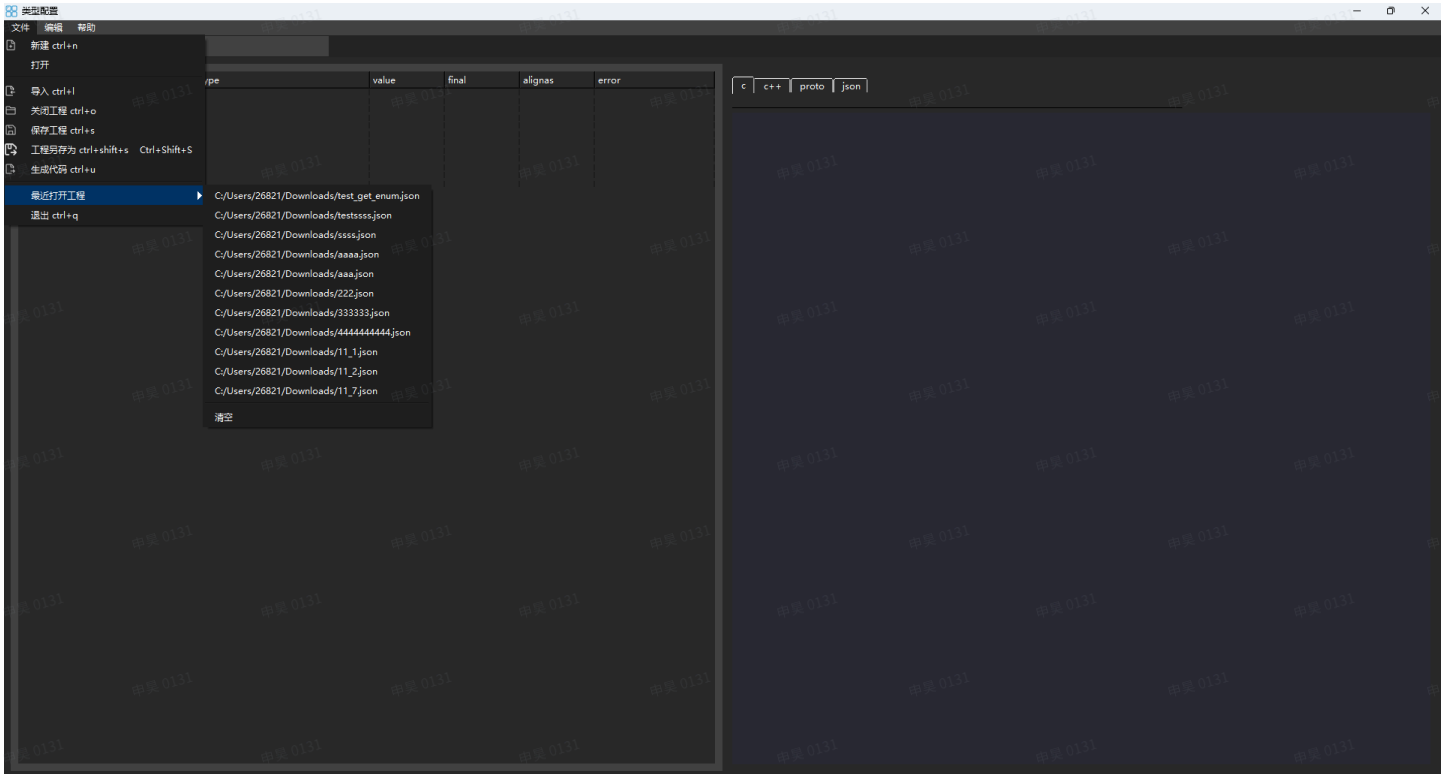
### 3.1 菜单栏区

菜单栏区包含文件、编辑、帮助、撤销等下拉菜单。用户可以使用该区域的菜单进行工程的创建/版本的获取/折叠/展开等多种操作。




**文件菜单：**

- **新建工程**：创建一个新的工程文件。
- **打开工程**：打开一个已存在的工程文件。
- **保存工程**：保存当前工程。
- **另存为**：将当前工程保存为一个新的文件。
- **导入**：从外部文件导入数据或配置。
- **关闭工程**：关闭当前打开的工程。
- **最近打开的工程**：列出最近打开的工程列表，便于快速访问。
- **生成代码**：生成类型配置好的对应代码文件。
- **错误检查**:检查当前工程是否有错误。
- **退出**：关闭应用程序。



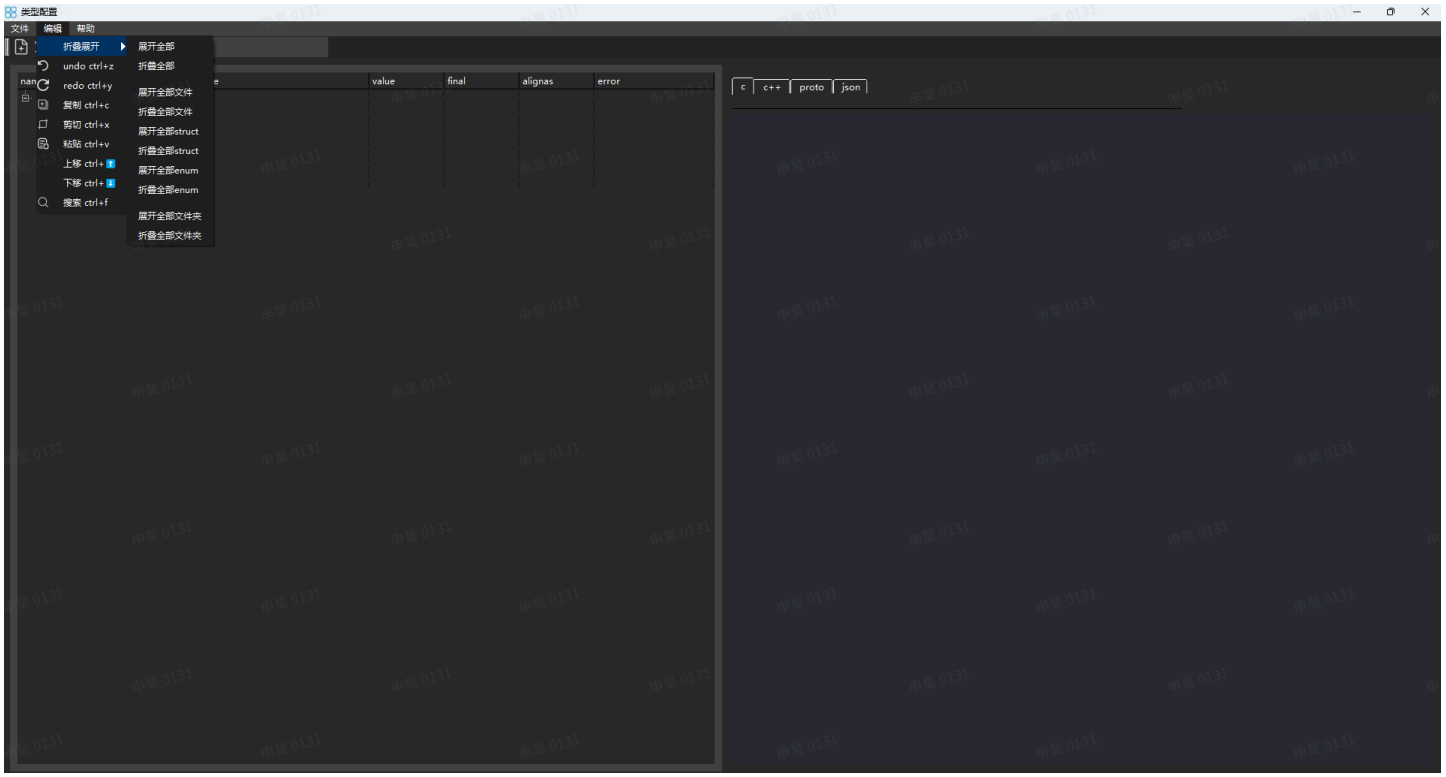

**编辑菜单：**

1. **Undo（撤销）**：撤销最后一次操作。
2. **Redo（重做）**：重做最后一次被撤销的操作。
3. **复制**：复制选中的文本或项目。
4. **剪切**：剪切选中的文本或项目到剪贴板。
5. **粘贴**：将剪贴板的内容粘贴到当前位置。
6. **上移**：将选中的项目或代码行向上移动。
7. **下移**：将选中的项目或代码行向下移动。
8. **搜索**：在当前项目中搜索特定文件。
9. **折叠展开**：管理项目结构的可见性。

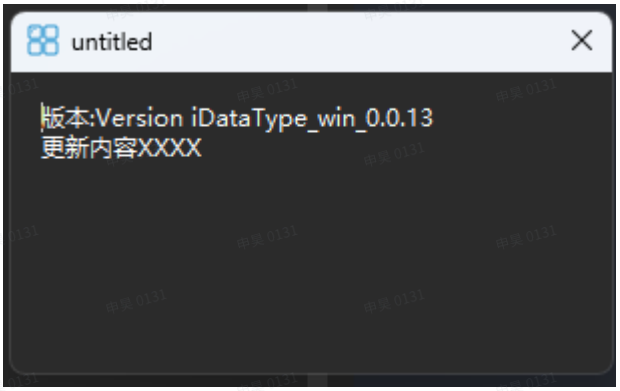
**折叠展开子菜单：**

1. **展开全部**：展开所有折叠的项目结构。
2. **折叠全部**：折叠所有展开的项目结构。
3. **展开全部文件**：展开所有文件。
4. **折叠全部文件**：折叠所有文件。
5. **展开全部struct**：展开所有结构体（struct）定义。
6. **折叠全部struct**：折叠所有结构体定义。
7. **展开全部enum**：展开所有枚举（enum）定义。
8. **折叠全部enum**：折叠所有枚举定义。

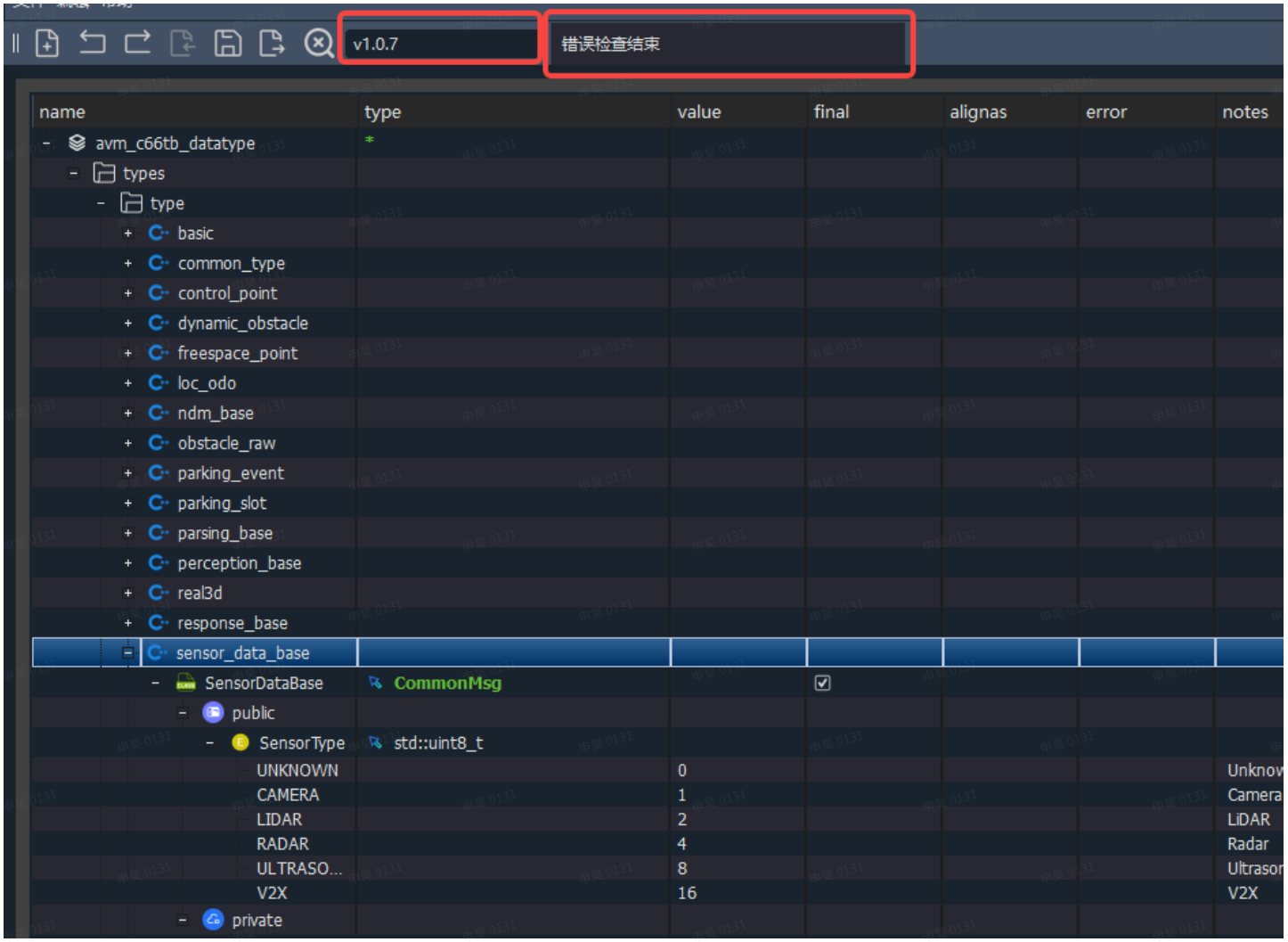
9. **展开全部文件夹**：展开所有项目中的文件夹。
10. **折叠全部文件夹**：折叠所有项目中的文件夹。




帮助下有获取版本，通过该方式来获取当前工具版本及其支持功能介绍。




提示数据类型的版本信息（版本信息可自己手动填写，此信息是供iview、iReplay读取），和log提示信息。




## 3.2 类型列表

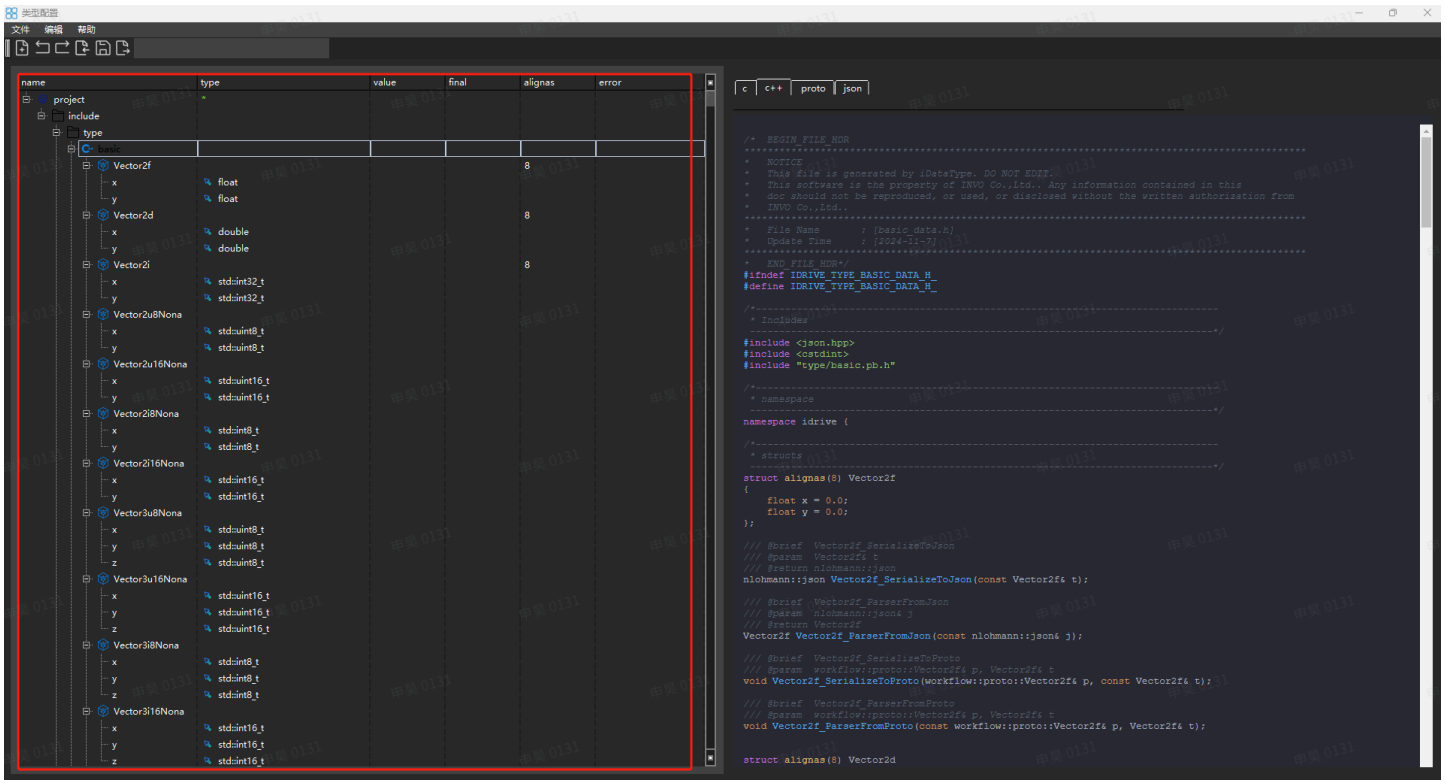
类型列表提供了一个清晰的结构化视图，显示项目中所有数据类型及其嵌套关系

**类型列表的内容：**

目录（Dir） / 文件（File） / 类（Class） / 结构体（Struct） / 枚举（Enum） / 变量（Var。

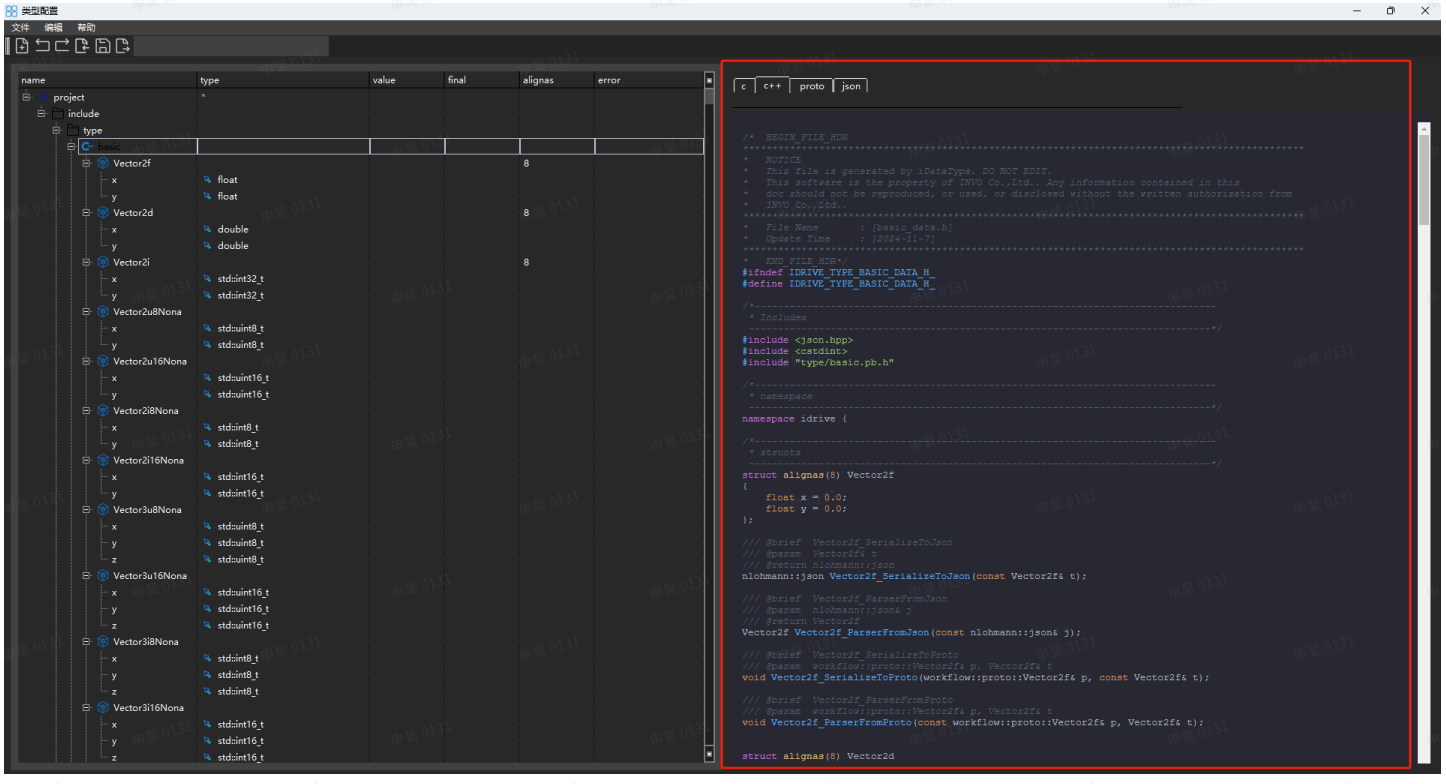
**嵌套关系：**

- **工程**：顶级容器，包含目录和文查看件。
- **目录**：可以包含其他目录和文件。
- **文件**：可以包含类、结构体、枚举和静态变量等类型定义。




## 3.3 预览区域

预览用户配置好的类型对应的代码，用户可以在此区域预览C/C++/PROTO/JSON这四种风格的代码。

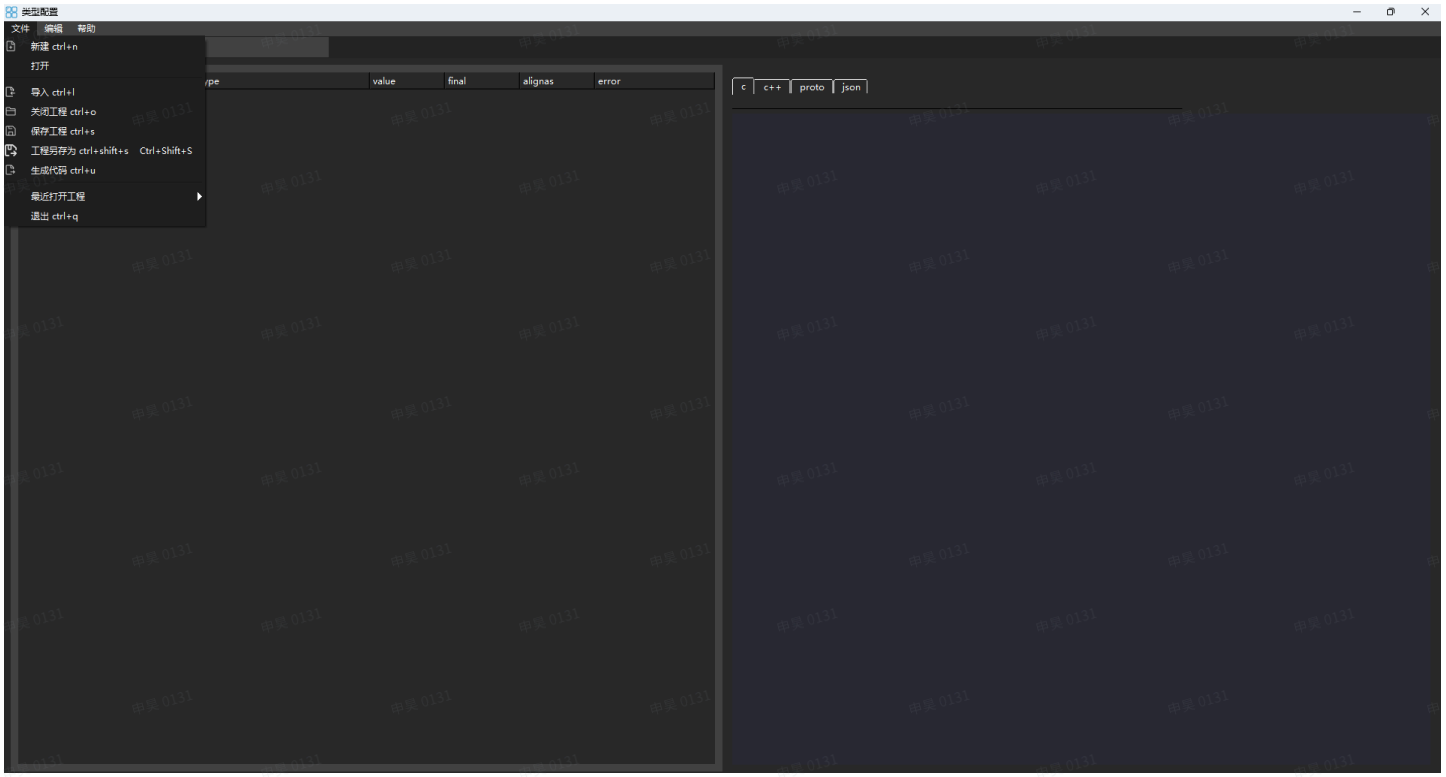



# 4. 功能介绍

## 4.1 新建工程

1. 在菜单栏中找到“**文件**”选项，并点击它。

1. 在下拉菜单中选择“**新建**”选项，或忽略以上第一条操作，直接点击菜单栏区的图标 
2. 在弹出的对话框中，选择工程存放的位置，用户可以浏览到合适的文件夹。
3. 自定义工程名，通常需要在指定的输入框中填写。
4. 确认无误后，点击“保存”按钮。
5. 完成新建空工程



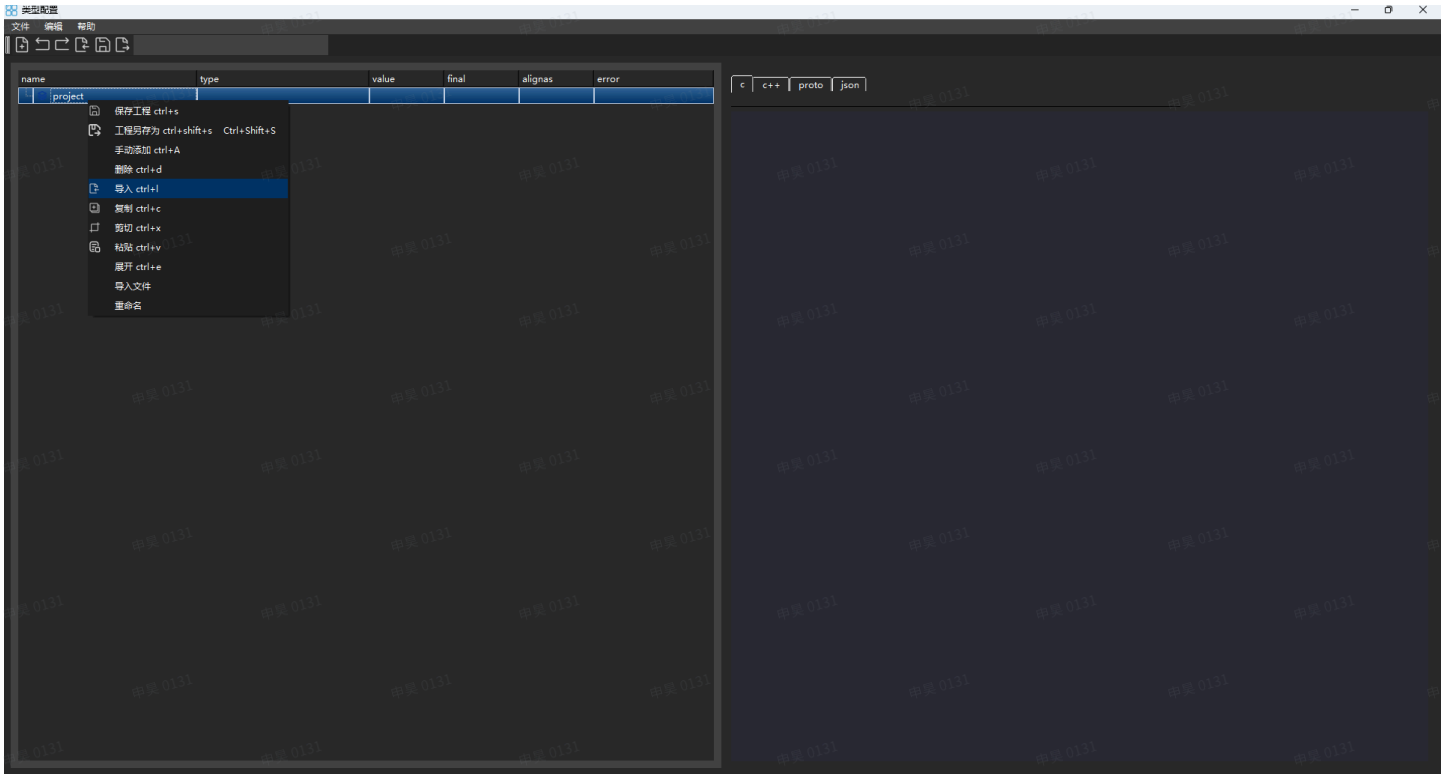

## 4.2 新增类型

建好工程后，可以选择导入或者手动添加类型：

### 4.2.1 导入目录

导入目录的操作步骤：

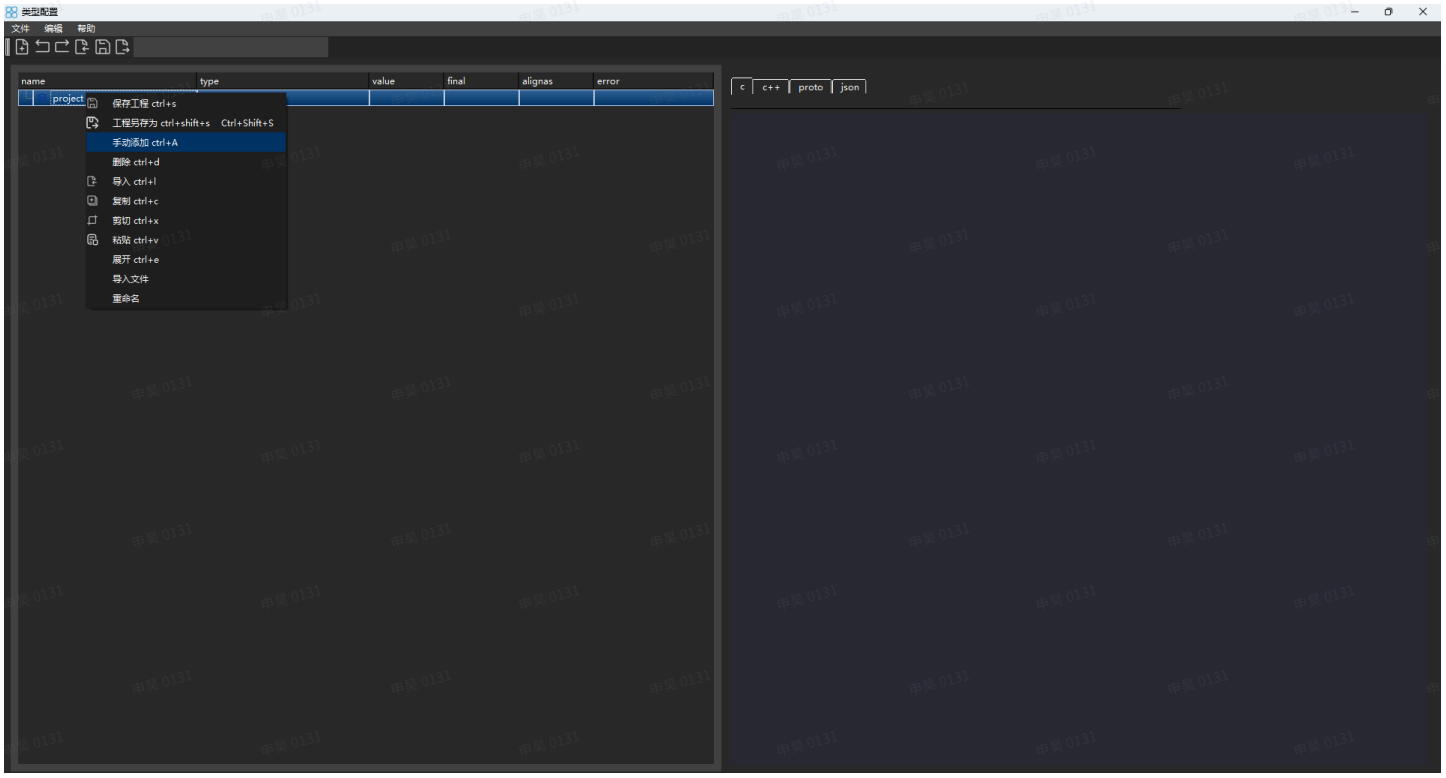
1. 定位工程：在开发环境中，找到并右键点击想要导入目录的工程名称。
2. 打开导入选项：在弹出的上下文菜单中，移动鼠标指针至“**导入**”选项，并点击它。
3. 选择目标目录：此时会弹出文件浏览器，浏览至想要导入的目录，选中该目录，然后点击“确定”或“打开”。
4. 导入过程提示：一旦选择了目录，系统会弹出一个提示框，显示导入进度。这个提示框会实时更新，告知导入操作的当前状态。
5. 统计信息：在导入过程中，提示框会统计并显示已导入的文件数量、结构体数量和枚举数量。
6. 完成导入：当导入操作完成后，提示框会通知导入操作已成功结束。此时，可以选择关闭提示框，并开始在工程中使用新导入的目录内容。



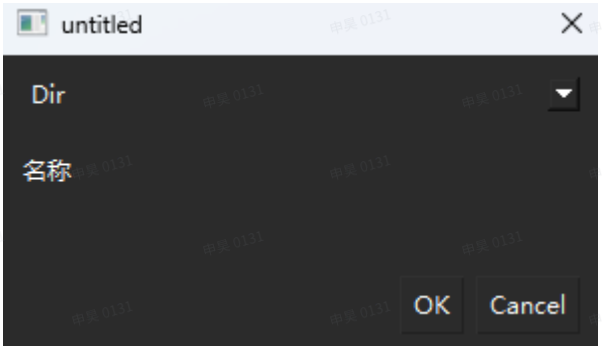

## 4.2.2 手动添加

除以上导入操作来完成新增类型，也可以手动新增数据类型：

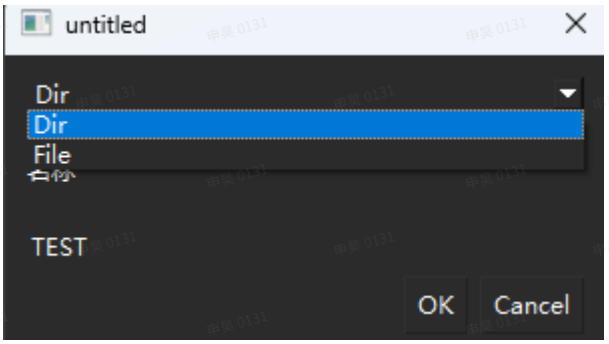
1. 右键点击工程名称，在弹出的上下文菜单中选择“手动添加”



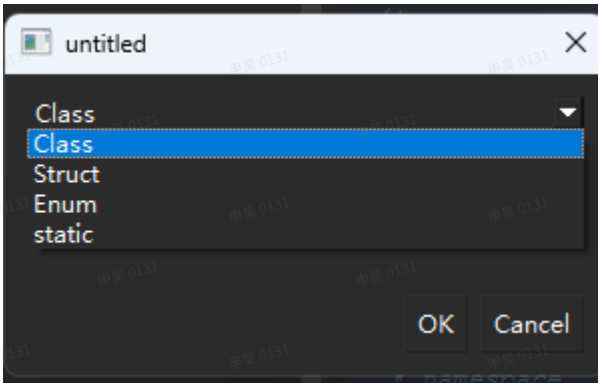

2. 在弹出的对话框或菜单中，可以看到可添加到工程中的不同类型的项目。这里通常会包括文件、文件夹等选项。
    a. **添加目录到工程：**在工程层级下，只能添加目录，输入目录名称并确认创建。




b. **在目录下添加文件：**在工程管理窗口中，右键点击刚才创建的目录，选择“手动添加”，根据提示框选择添加的类型，例如“文件”或“目录”，输入目录或文件名称并确认创建。




c. **在文件下添加类型：**在工程管理窗口中，右键点击刚才创建的文件，选择“手动添加”，根据项目需求，选择以下类型之一（Class / Struct / Enum / Static），随后输入对应名称，并确认创建。

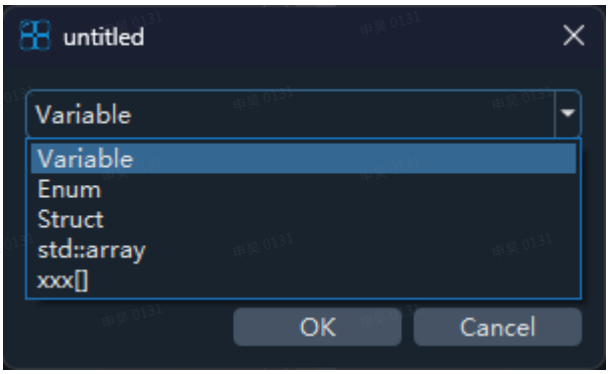



通过以上描述的操作不断重复，可以在工程中创建多个目录/文件层级。同时也可以通过菜单栏的复制粘贴操作简化此过程。

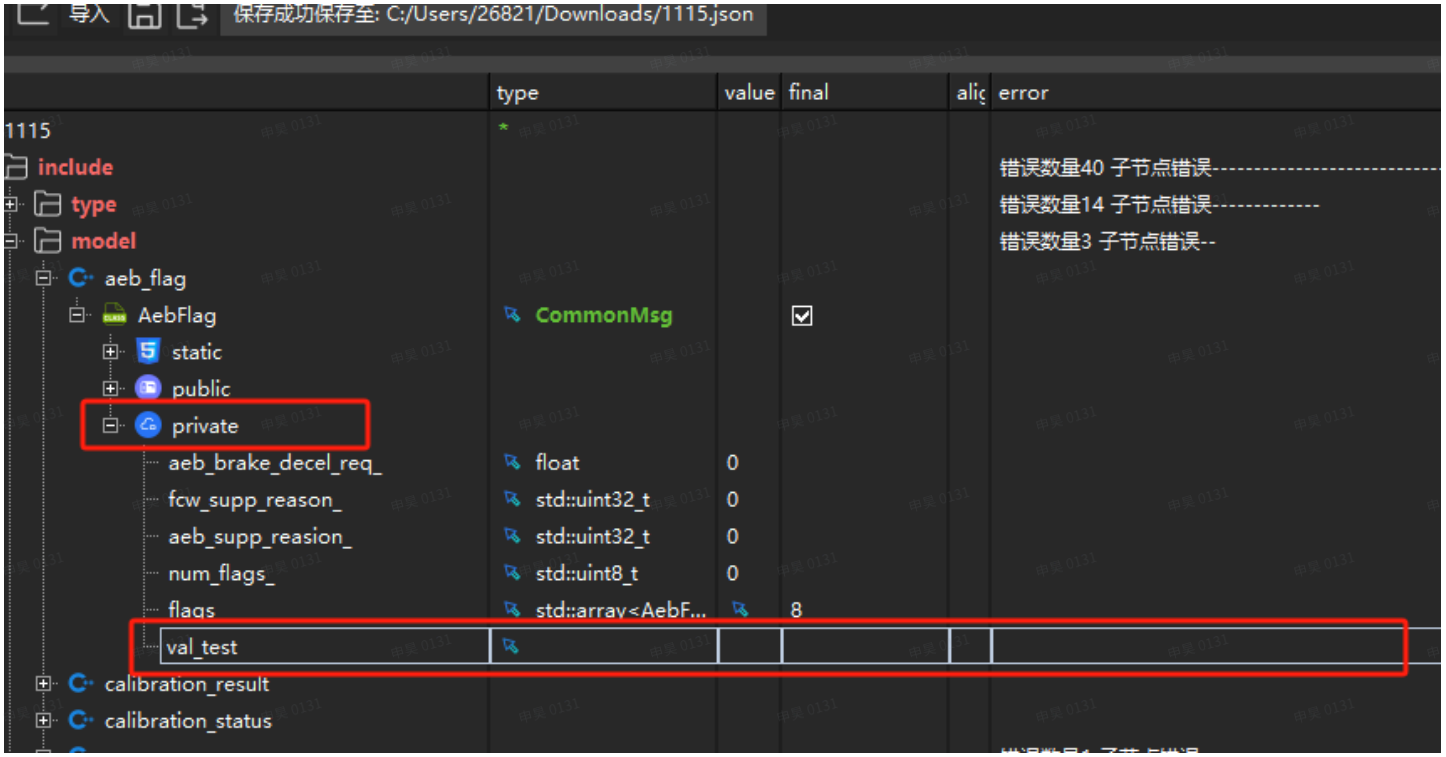
下面将详细介绍如何添加详细类型：

#### 4.2.2.1 类内添加普通变量

1. 右键类下的”private“选择手动添加，选择Variable, 并自定义变量名，点击OK

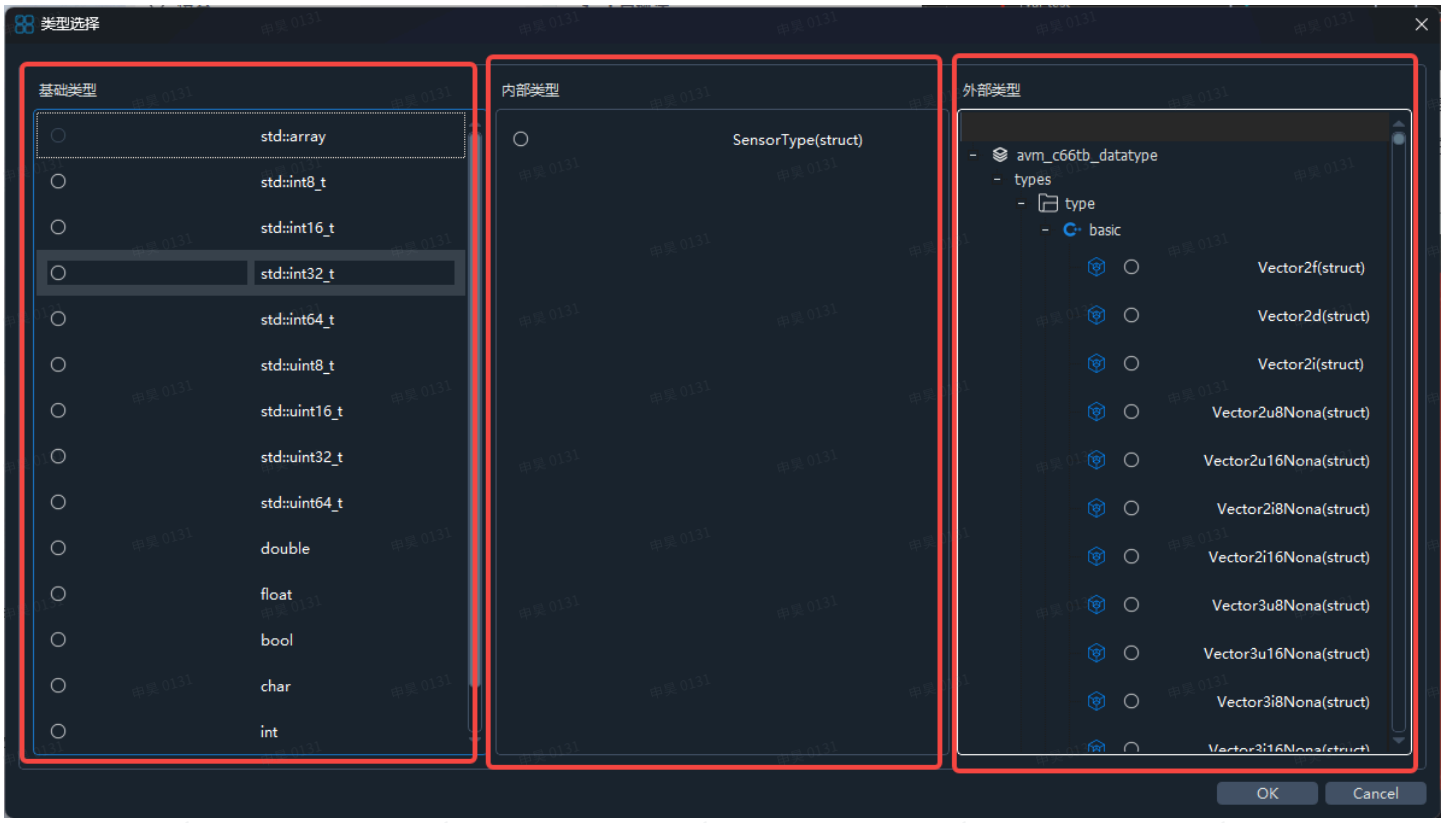



2. 点击OK后，效果如下：

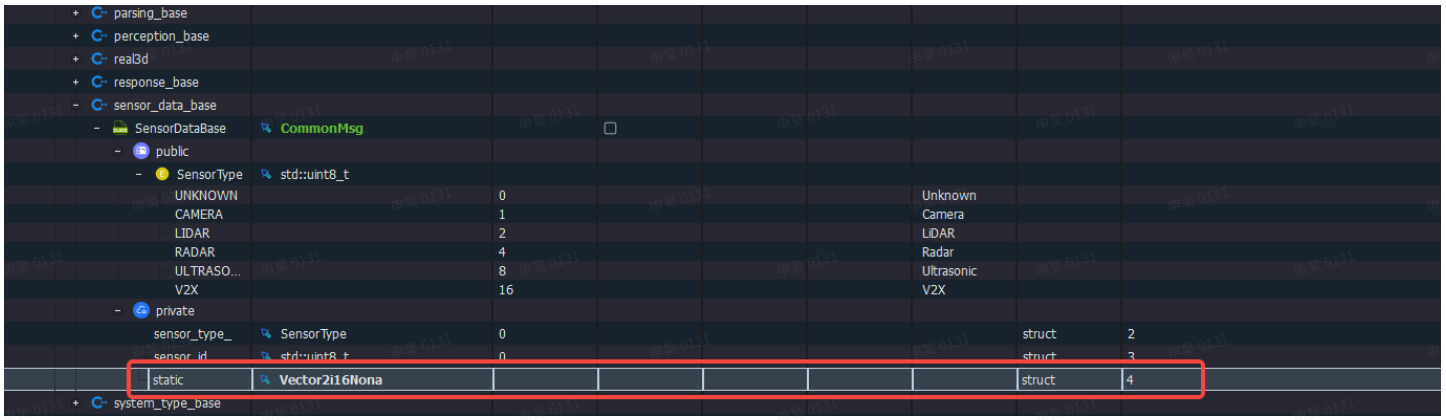



随后点击新添加变量对应的type栏处，在弹出的弹框中，选择该变量的类型。

最左侧一栏是非本类中定义的类型，中间一栏是基础类型，右侧一栏是本类中定义的类型（结构体/枚举）。




勾选对应的类型后，点击OK，完成类型选择。以下是点击后的效果。



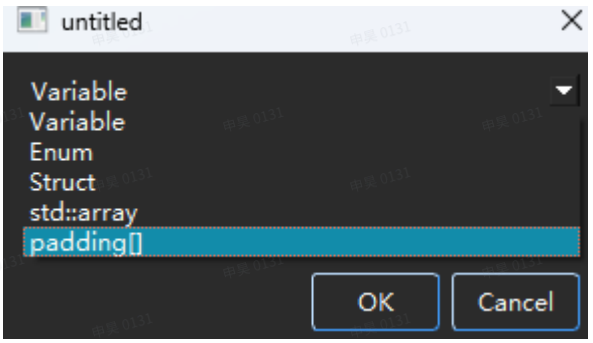

3. 随后点击value栏，设置默认值，此操作不是必须的。

### 4.2.2.2 添加c风格数组

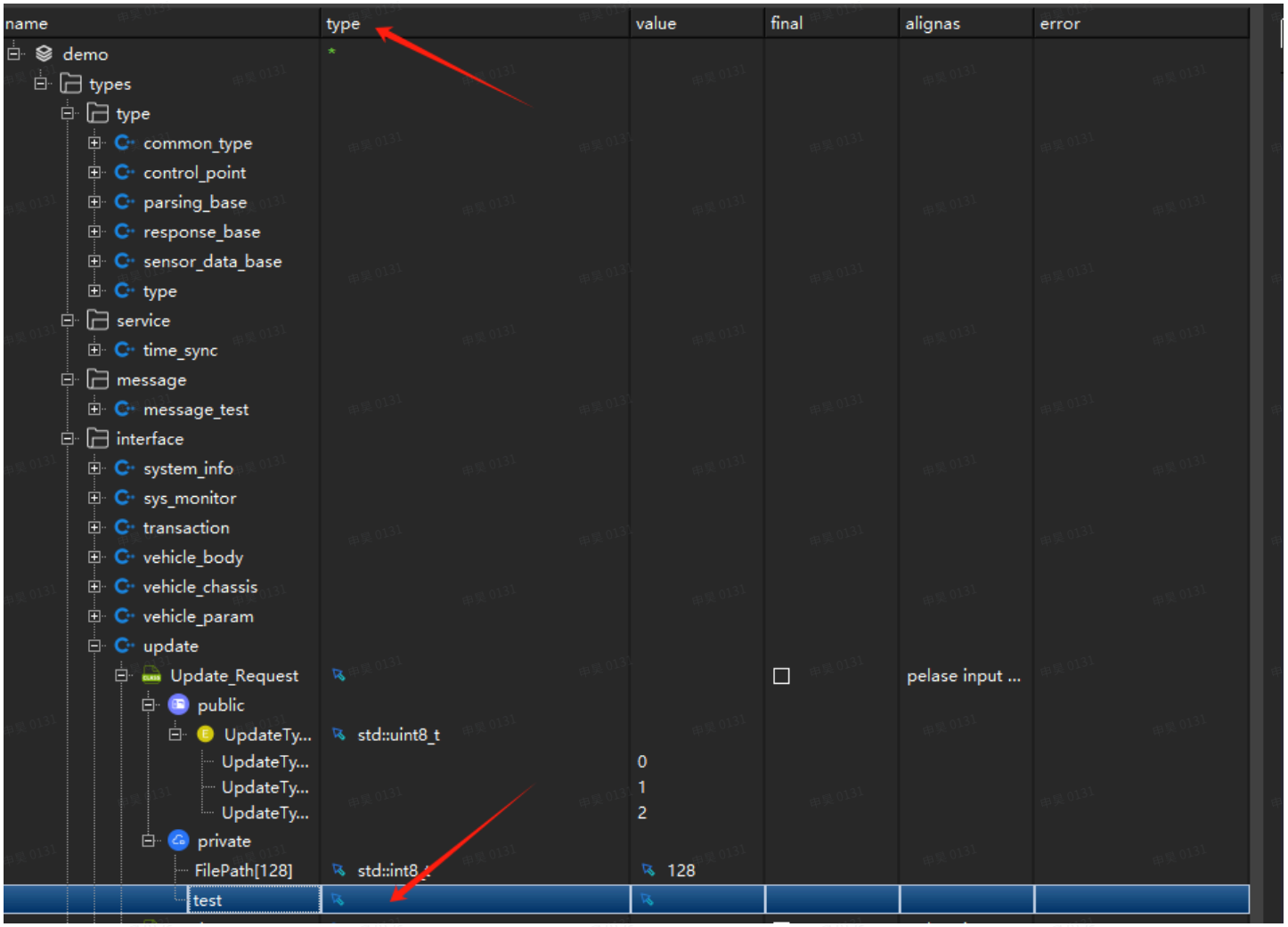
例如int padding[20];

操作如下：

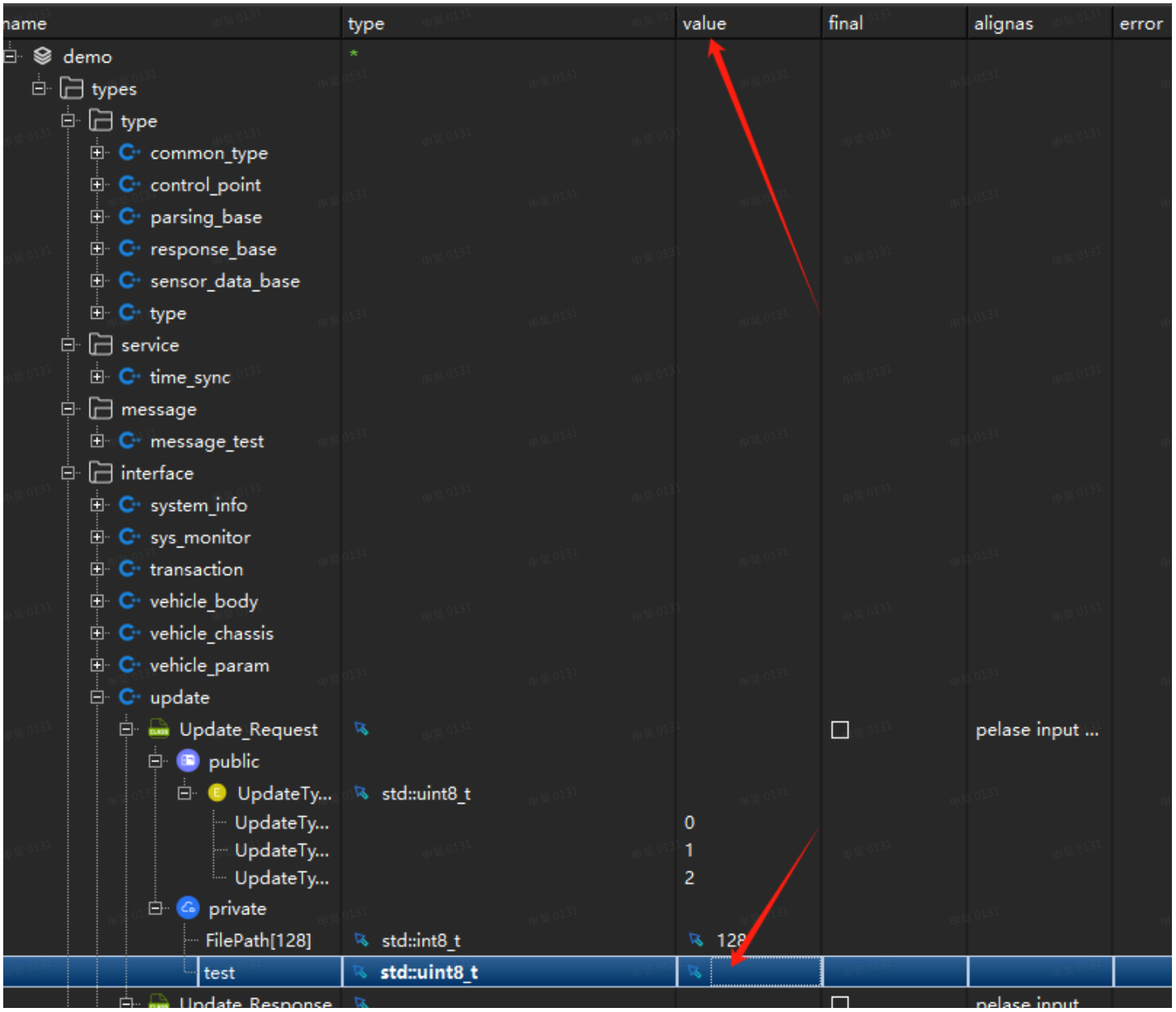
1. 点击手动添加，选择padding[], 并自定义类型名



2.随后点击该类型对应type栏，通过此方式来选择数组类型




3.随后点击该类型的value栏，通过此方式来选择数组的大小

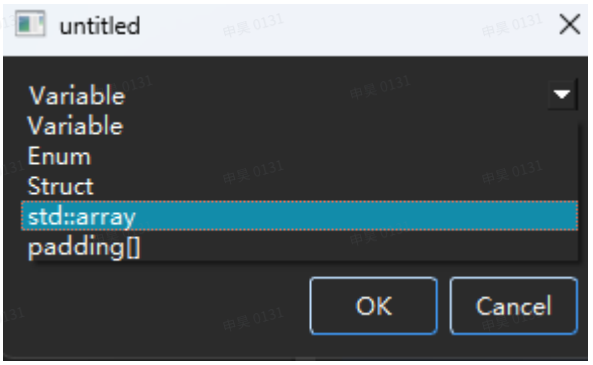



随后在跳出的弹框中选择大小，可以选择静态变量，也可以选择自定义值

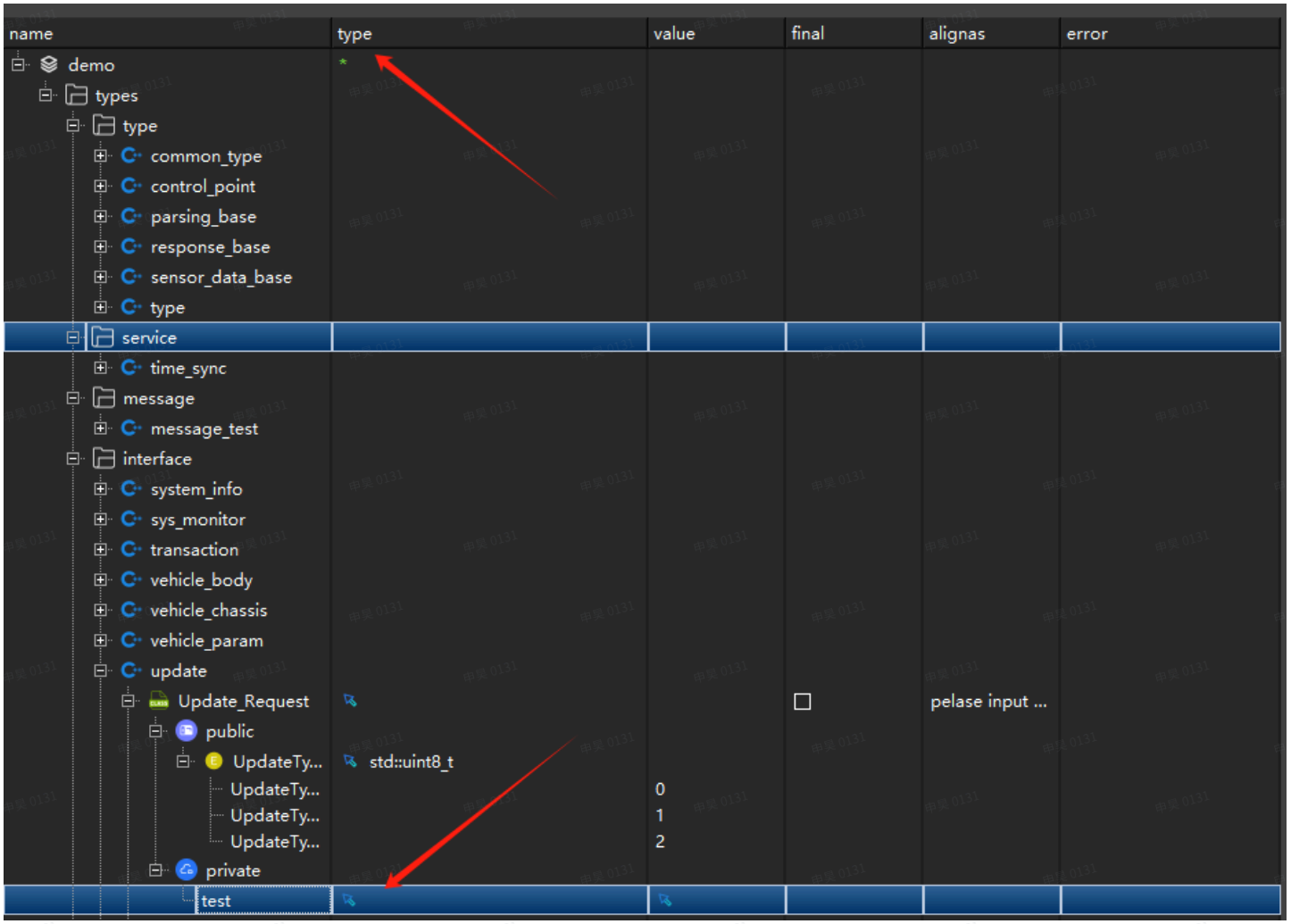



#### 4.2.2.3 添加c++风格一维数组

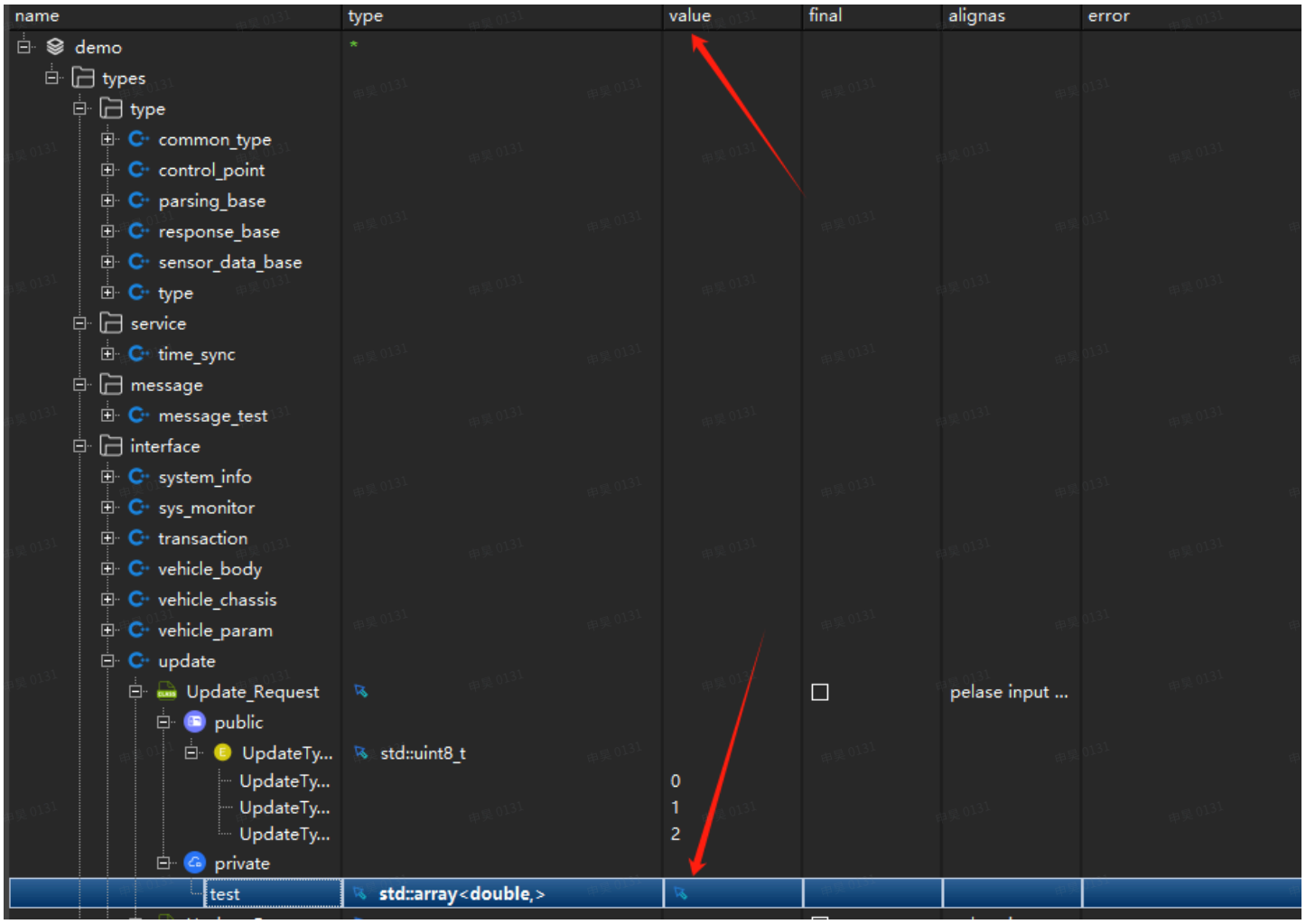
1. 点击手动添加，选择std::array, 并自定义变量名




2. 随后点击该变量对应的type栏，选择array类型




3. 随后点击该变量对应的value栏，选择array大小




4. 选择后，至此结束




5. 添加c++数组的真实大小:





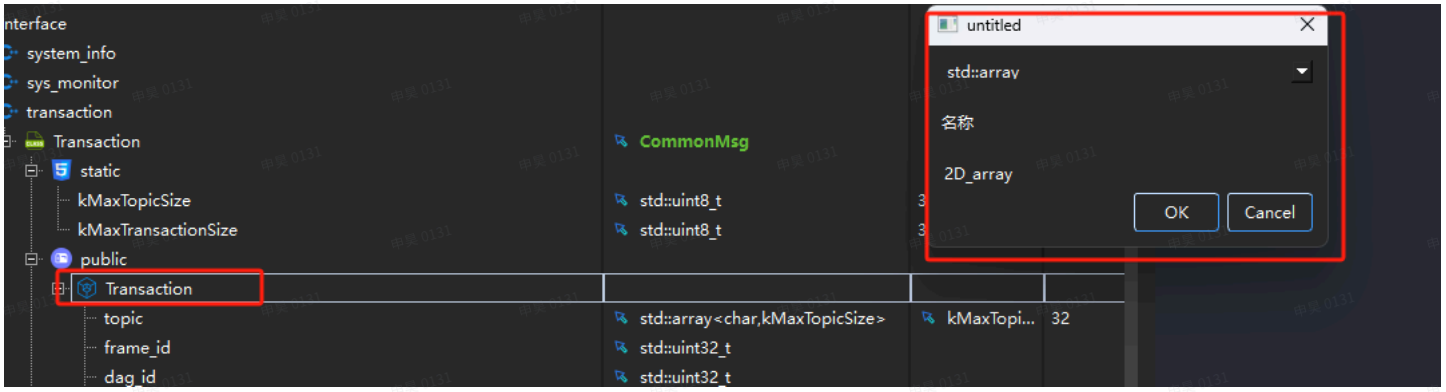


选择当前同级目录下的变量。添加成功后如下。

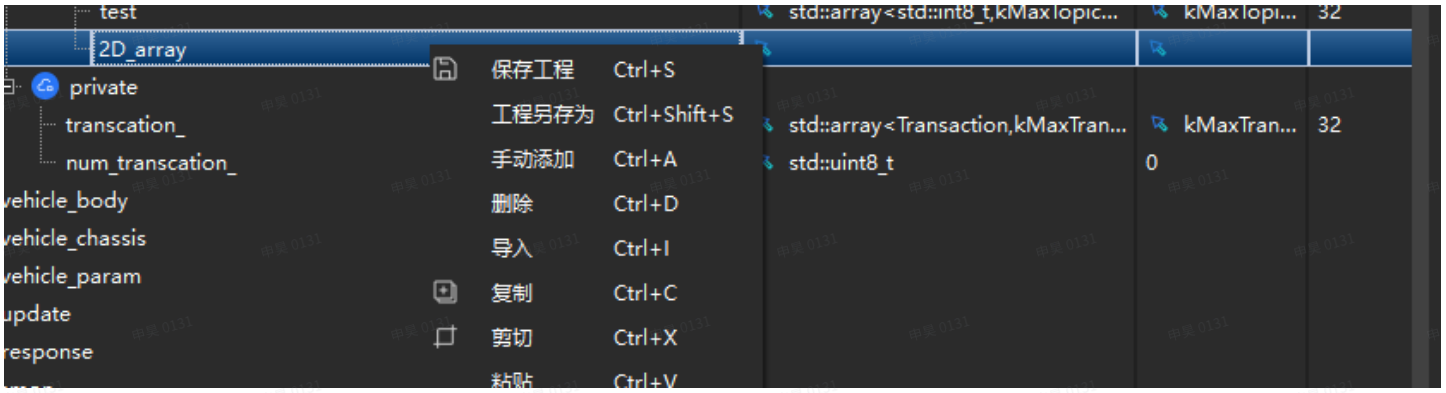



### 4.2.2.4 添加c++风格二维数组

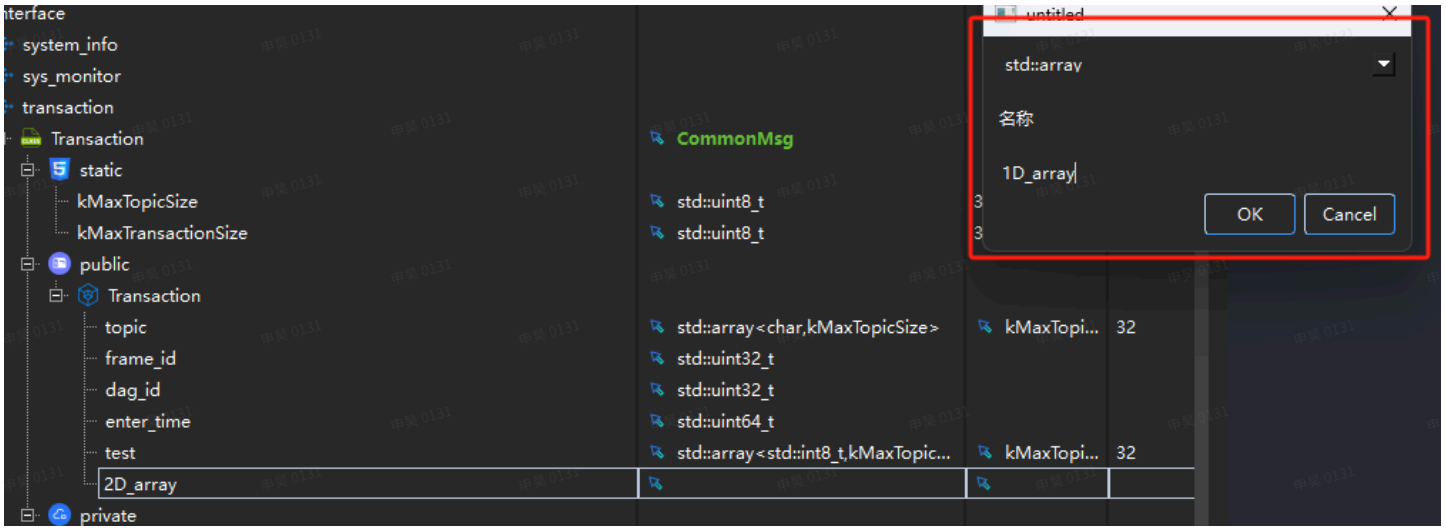
1. 右键点击所要添加的父级节点，手动添加选择std::array, 并自定义数组名字，以下实例中使用2D_array作为二维数组的名称。




2. 右键点击二维数组名称处，选择手动添加

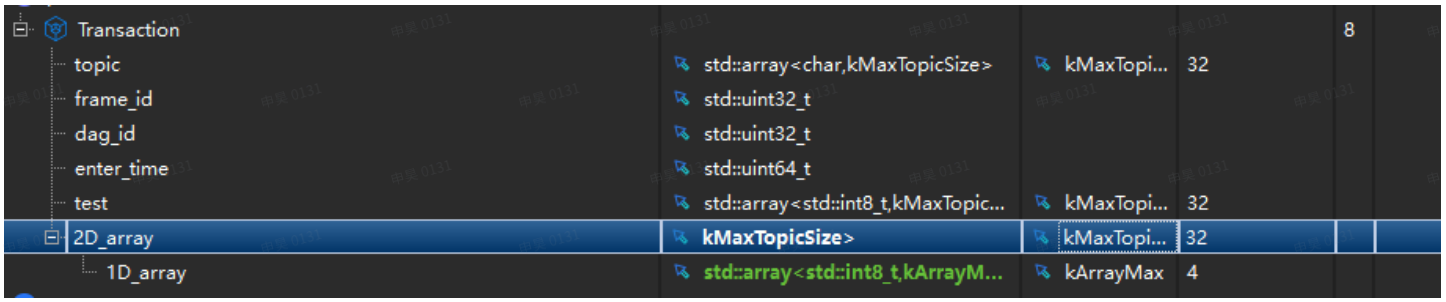



选择std::array 并自定义数组名




3. 点击一维数组的type栏处，选择一维数组里类型

4. 点击一维数组的value栏处，选择一维数组里的大小
5. 点击二维数组的value栏处，选选择二维数组的大小
6. 至此，完成二维数组的添加，效果如下




代码如下：

```
struct alignas(8) Transaction
{
    std::array<char, kMaxTopicSize> topic;
    std::uint32_t frame_id;
    std::uint32_t dag_id;
    std::uint64_t enter_time;
    std::array<std::int8_t, kMaxTopicSize> test;
    std::array<std::array<std::int8_t, kArrayMax>, kMaxTopicSize> 2D_array;
};
```

### 4.2.2.5 类内添加string变量

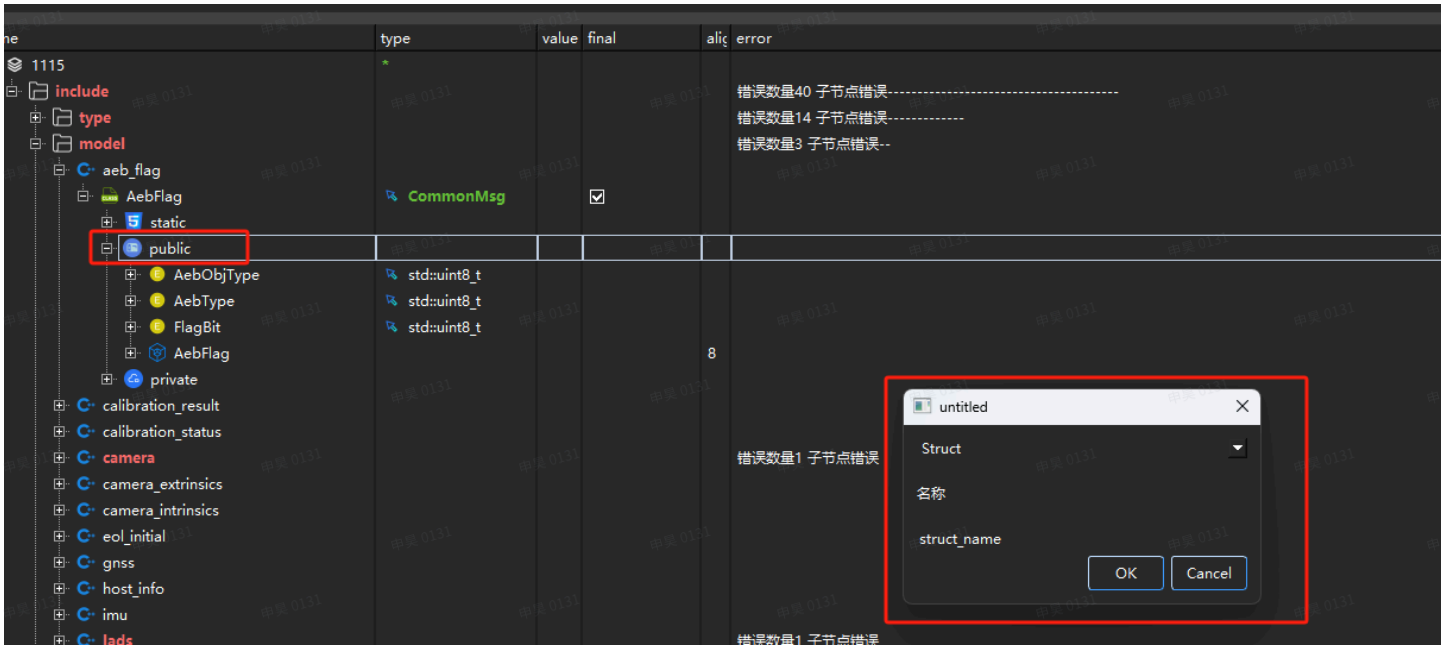
两种方式来生成string类型的普通变量

1. 添加char var_name[size]
2. 添加std::array<char, size> var_name

以上两种方式参考4.2.2.2 / 4.2.2.3 ，仅在选择数组类型时，选择char即可。

### 4.2.2.6 类内添加struct

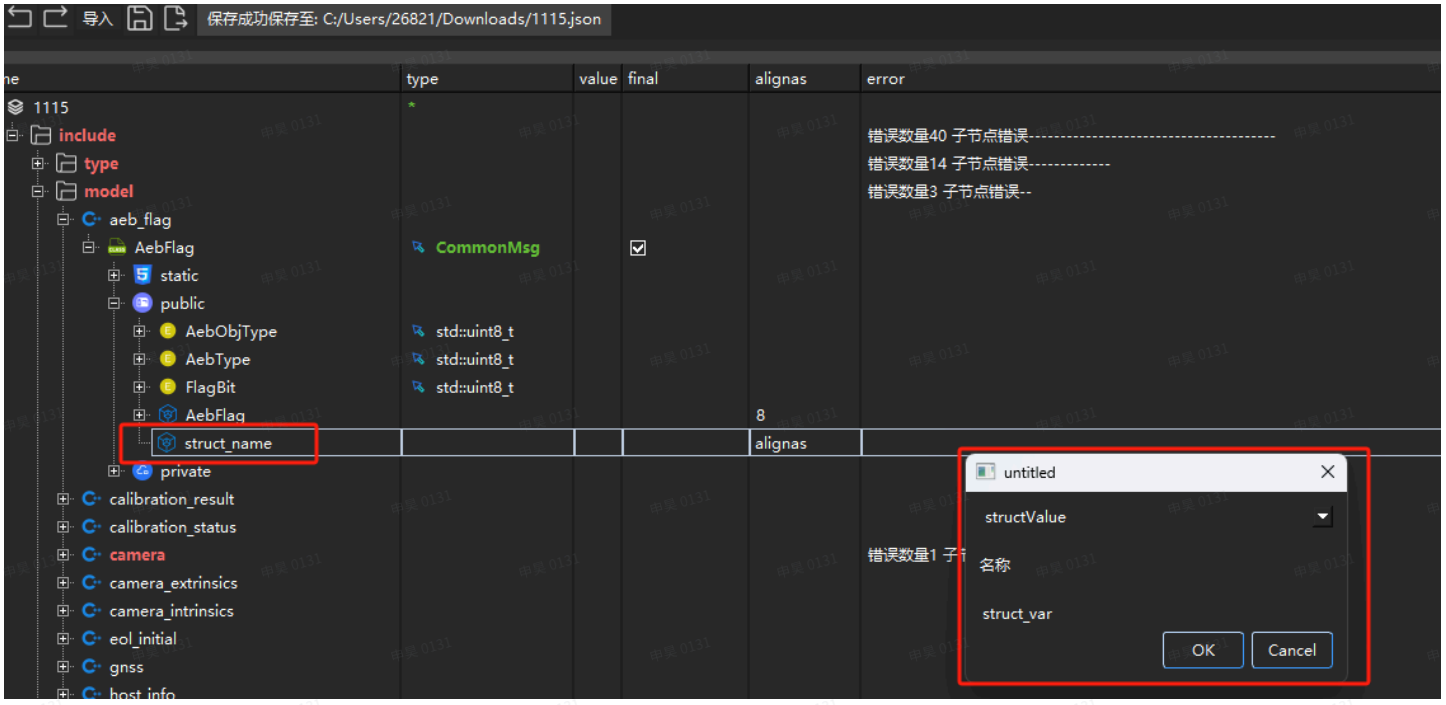
1. 右键类的public处，随后在弹框中选择Struct, 自定义结构体名字，点击OK。



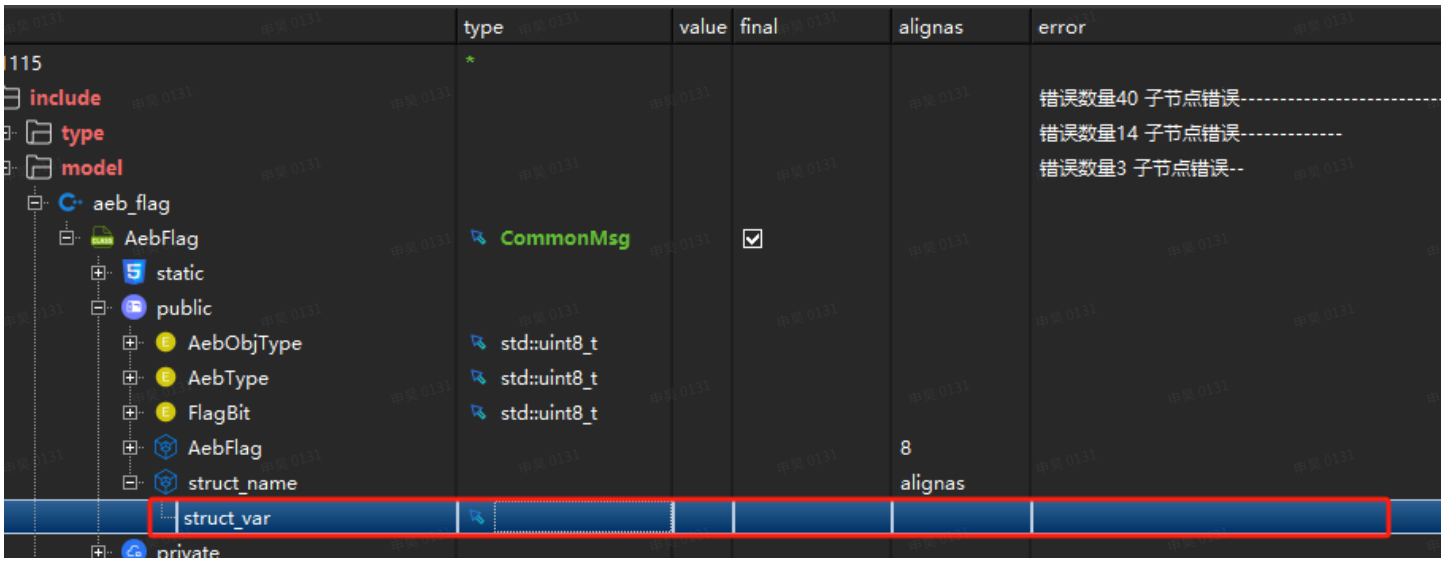

2. 随后点击生成的struct对应的alignas处，自定义对齐字节数， 默认为8




3. 随后右键点击生成的struct, 选择添加structValue类型,点击OK




4. 随后点击生成的struct成员的type栏选择对应类型




## 4.2.2.7 类内添加enum

1. 右键点击类的public处，选择添加为enum,并自定义enum name, 并点击OK



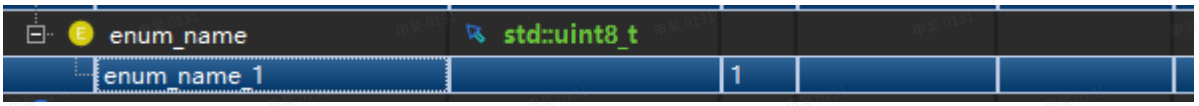

2. 在生成的enum栏，点击对应type栏，选择枚举类的底层类型，默认选择std::uint8_t




3. 右键点击enum, 添加enumValue, 自定义枚举值名字，点击OK




4. 点击枚举值对应的value栏，设置枚举值的显示值。




#### 4.2.2.8 添加注释

当前支持在三种地方添加注释，方式如下：

1. 选中枚举内的值，找到notes列，添加注释
2. 选中结构体内的值，找到notes列，添加注释
3. 选中私有成员，找到notes列，添加注释

## 4.3 错误检查

再点击生成代码时，类型工具检查代码中的类型错误（类类型工具不支持模板类），确保后续生成bi的代码能够被编译器正确解析。根据类型工具提供的错误提示，找到出错的位置，并修正类型错误；

如果工程中有重名的类、结构体、枚举，类型工具会报错，检查项目中重名的元素，并修改它们的名字，确保每个元素在作用域内是唯一的；

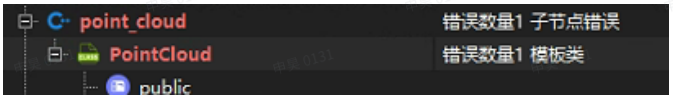
错误提示种类：

### 4.3.1 重名错误

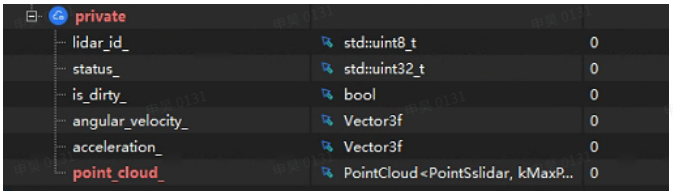
a. 类名冲突
b. 类名与类外的结构体/枚举冲突
c. 同一个类里的成员变量名冲突
d. 同一个类里的枚举名冲突 / 类外的枚举名冲突
e. 同一个类里的结构体名冲突 / 类外的结构体名冲突
f. 与系统函数重名，目前检测重名 max  min

### 4.3.2 模板类错误

1. 模板类定义（当前版本不支持模板类）




2. 模板类使用



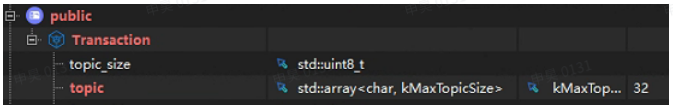
### 4.3.3 结构体成员定义错误

a. 同时有std::array ArrayA , int ArrayA_size

b. `STRUCT_NODE` 类型节点不能循环引用其他结构体，结构体互相引用，造成死循环

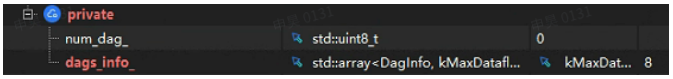
### 4.3.4 类私有成员定义错误

1. 同时有std::array ArrayA , int ArrayA_size会提示错误（解决方法：删除int ArrayA_size）




2. 有std::array ArrayA, 没有int num_ArrayA

（解决方法：添加成员int num_ArrayA）

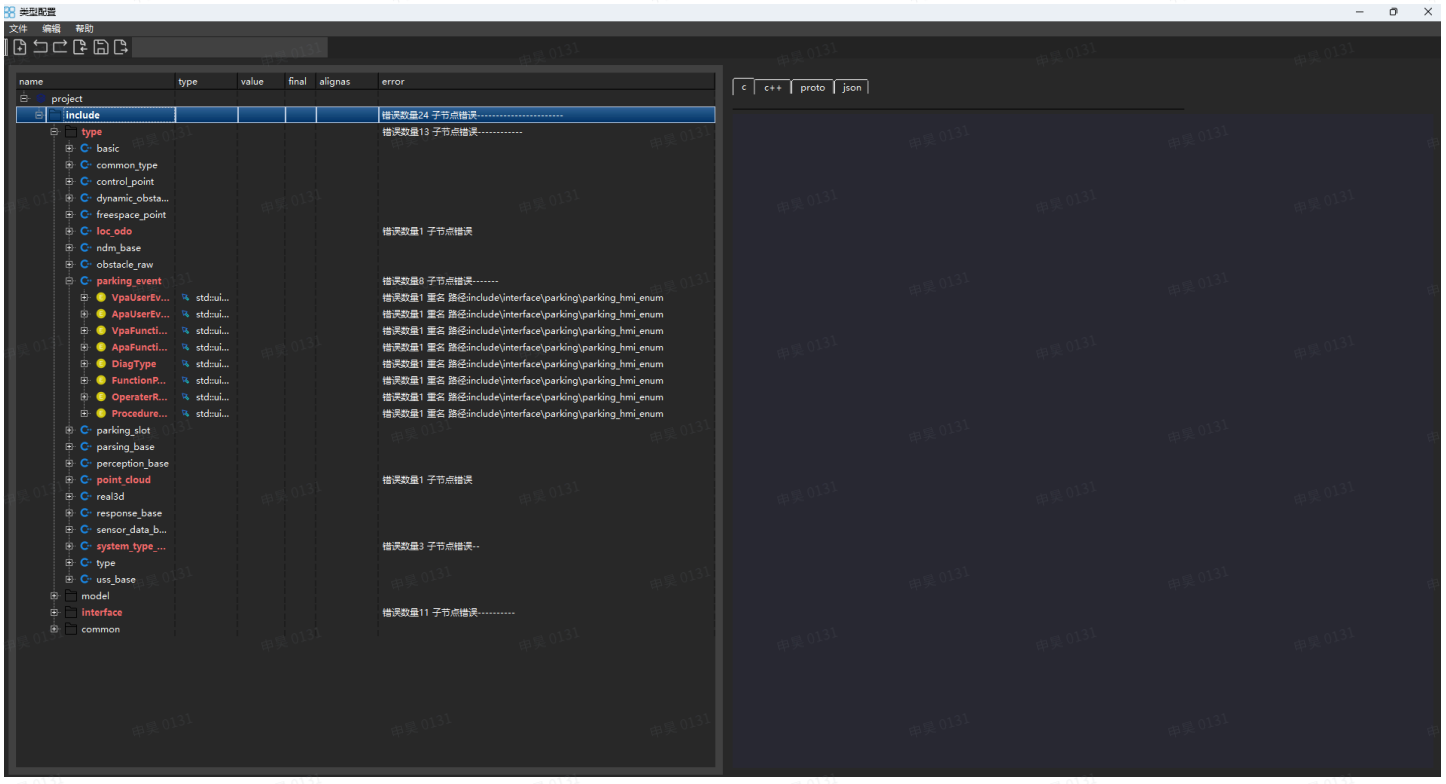



3. 导入外部文件，私有成员有静态变量定义

（解决方法：删除private下的静态变量定义）




如果工程中有资源文件缺失或路径错误，类型工具也会给出提示。




当修改完代码后CTRL+S，如果类型列表区域不再飘红，这意味着所有语法错误和类型问题都已经被修复。此时，可以通过点击保存或者使用快捷键CTRL+S方式保存更改。

### 4.3.5 数组相关检查

- 对于 `ARRAY_NODE` 类型节点，同级下不能存在名为 `{数组名}_size` 的成员变量（第0列）
- 对于 `ARRAY_NODE` 类型节点，如果父节点是 `PRIVATE_NODE`，则同级下必须存在名为 `num_{数组名}` 的成员变量（第0列）
- `ARRAY_NODE` 类型节点必须指定 `size_var`
- `ARRAY_NODE_C` 类型节点的名称（第0列）不能以 `[` 开头
- `CLASS_VARIABLE_NODE` 和 `STRUCT_VALUE` 类型节点的名称（第0列）不能包含 `[` 或 `]`，只有数组类型可以包含

### 4.3.6 类继承检查规则

- `CLASS_NDOE` 类型节点的继承类（第1列）不能等于类名（第0列），继承类不能为本类
- `CLASS_NDOE` 类型节点不能是模板类

### 4.3.7 类型检查规则

1. 以下节点类型的类型字段（第1列）不能为空：
   - `STRUCT_VALUE`
   - `ENUM_NODE`
   - `CLASS_VARIABLE_NODE`
   - `ARRAY_NODE`
   - `ARRAY_NODE_C`
2. `CLASS_VARIABLE_NODE` 和 `STRUCT_VALUE` 类型节点的类型字段（第1列）不能包含 `<`（即不能使用模板类类型）
3. 如果类型字段（第1列）包含 `::` 且不包含 `std`，则类名必须带 `Data` 后缀

### 4.3.8 枚举相关检查规则

- `ENUM_VALUE` 类型节点的名称（第0列）不能是 `MAX`、`MIN`、`max`、`min`
- 如果 `ENUM_VALUE` 类型节点的值（第2列）为 `0`，则它必须是父节点的第一个子节点（索引为0）
- 如果枚举值（第2列）为负数，但枚举类型（父节点的第1列）包含 `uint`（无符号类型），则报错

### 4.3.9 文件夹相关检查规则

- `DIR_NODE` 类型节点的名称（第0列）不能包含下划线 `_`

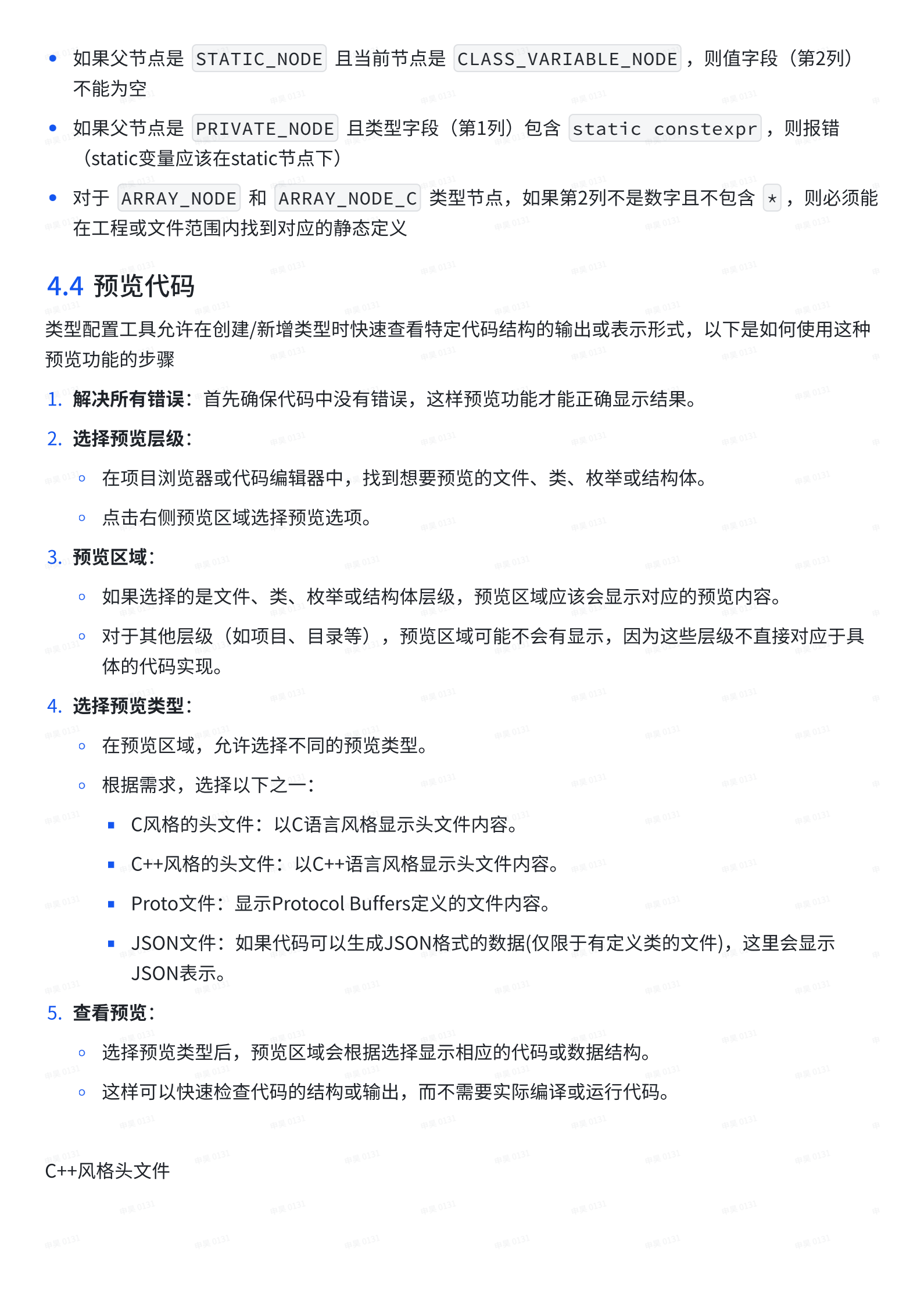
### 4.3.10 静态变量相关检查规则

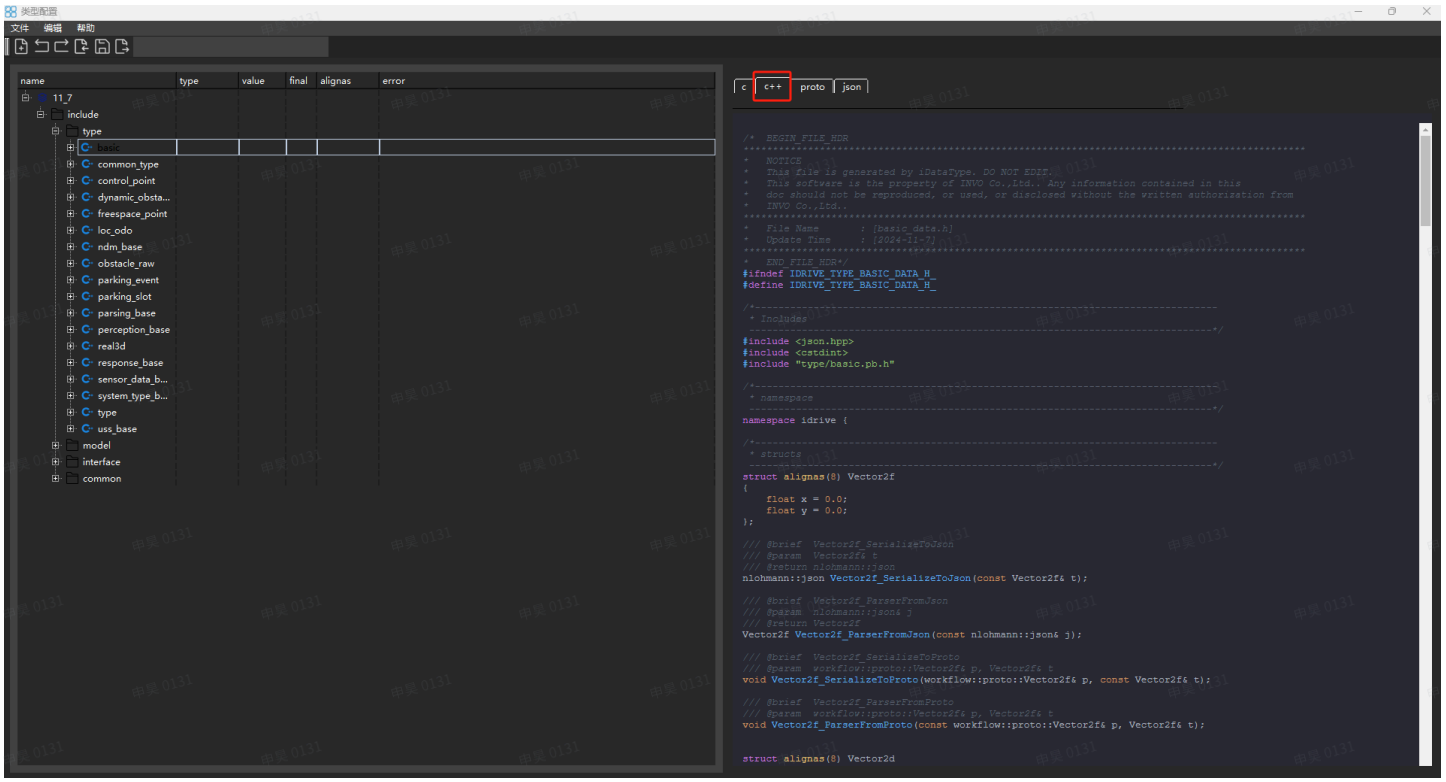
- 如果父节点是 `STATIC_NODE` 且当前节点是 `CLASS_VARIABLE_NODE`，则值字段（第2列）不能为空
- 如果父节点是 `PRIVATE_NODE` 且类型字段（第1列）包含 `static constexpr`，则报错（static变量应该在static节点下）
- 对于 `ARRAY_NODE` 和 `ARRAY_NODE_C` 类型节点，如果第2列不是数字且不包含 `*`，则必须能在工程或文件范围内找到对应的静态定义

## 4.4 预览代码

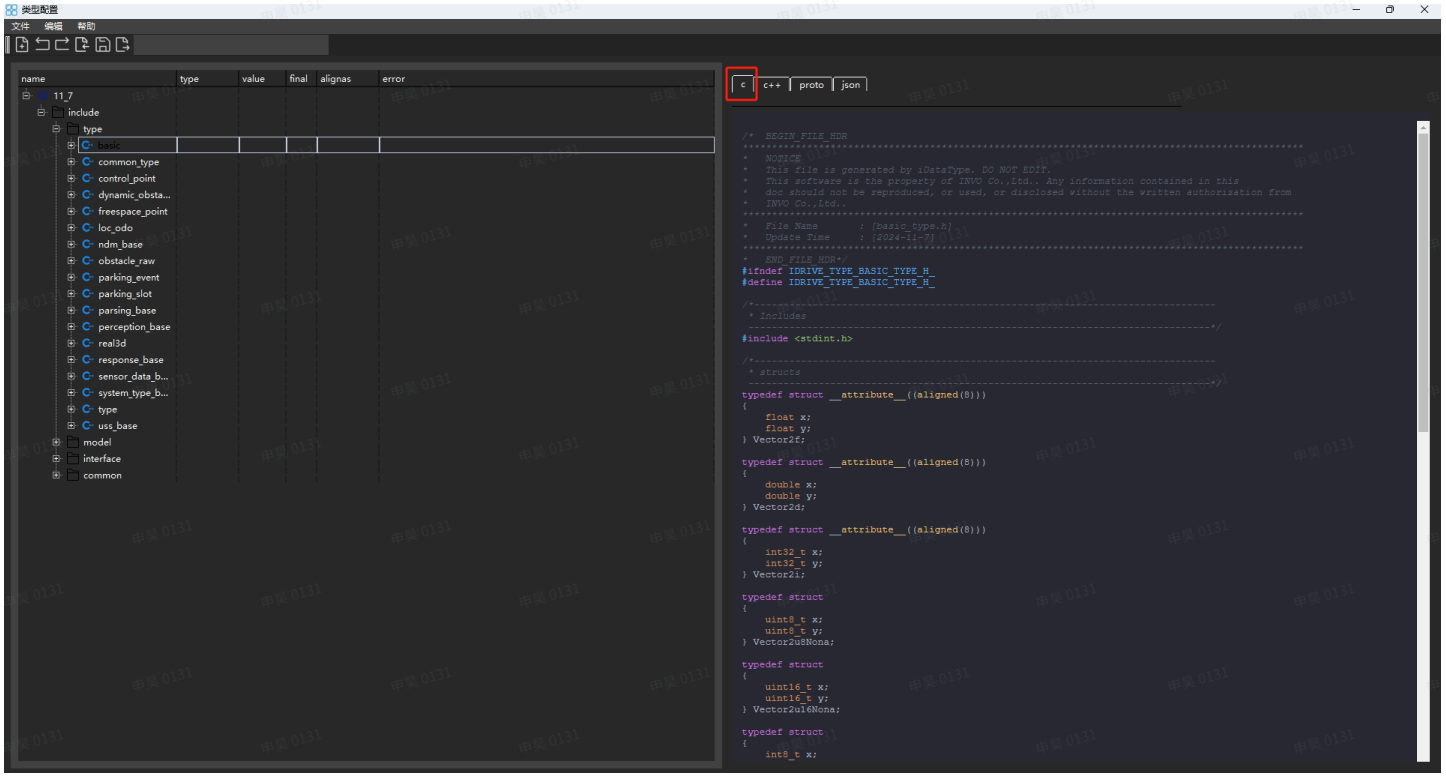
类型配置工具允许在创建/新增类型时快速查看特定代码结构的输出或表示形式，以下是如何使用这种预览功能的步骤

1. **解决所有错误**：首先确保代码中没有错误，这样预览功能才能正确显示结果。
2. **选择预览层级**：
    - 在项目浏览器或代码编辑器中，找到想要预览的文件、类、枚举或结构体。
    - 点击右侧预览区域选择预览选项。
3. **预览区域**：
    - 如果选择的是文件、类、枚举或结构体层级，预览区域应该会显示对应的预览内容。
    - 对于其他层级（如项目、目录等），预览区域可能不会有显示，因为这些层级不直接对应于具体的代码实现。
4. **选择预览类型**：
    - 在预览区域，允许选择不同的预览类型。
    - 根据需求，选择以下之一：
        - C风格的头文件：以C语言风格显示头文件内容。
        - C++风格的头文件：以C++语言风格显示头文件内容。
        - Proto文件：显示Protocol Buffers定义的文件内容。
        - JSON文件：如果代码可以生成JSON格式的数据(仅限于有定义类的文件)，这里会显示JSON表示。
5. **查看预览**：
    - 选择预览类型后，预览区域会根据选择显示相应的代码或数据结构。
    - 这样可以快速检查代码的结构或输出，而不需要实际编译或运行代码。

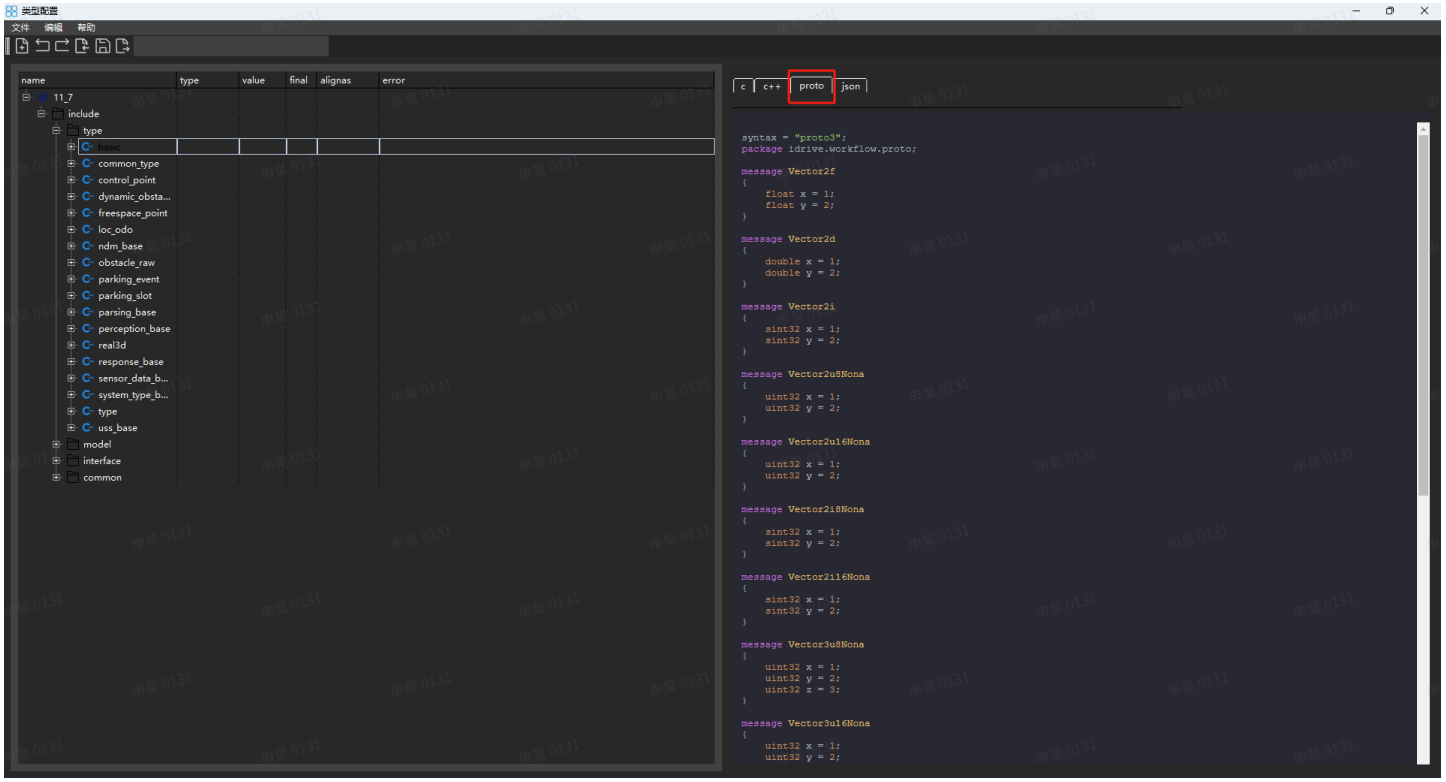




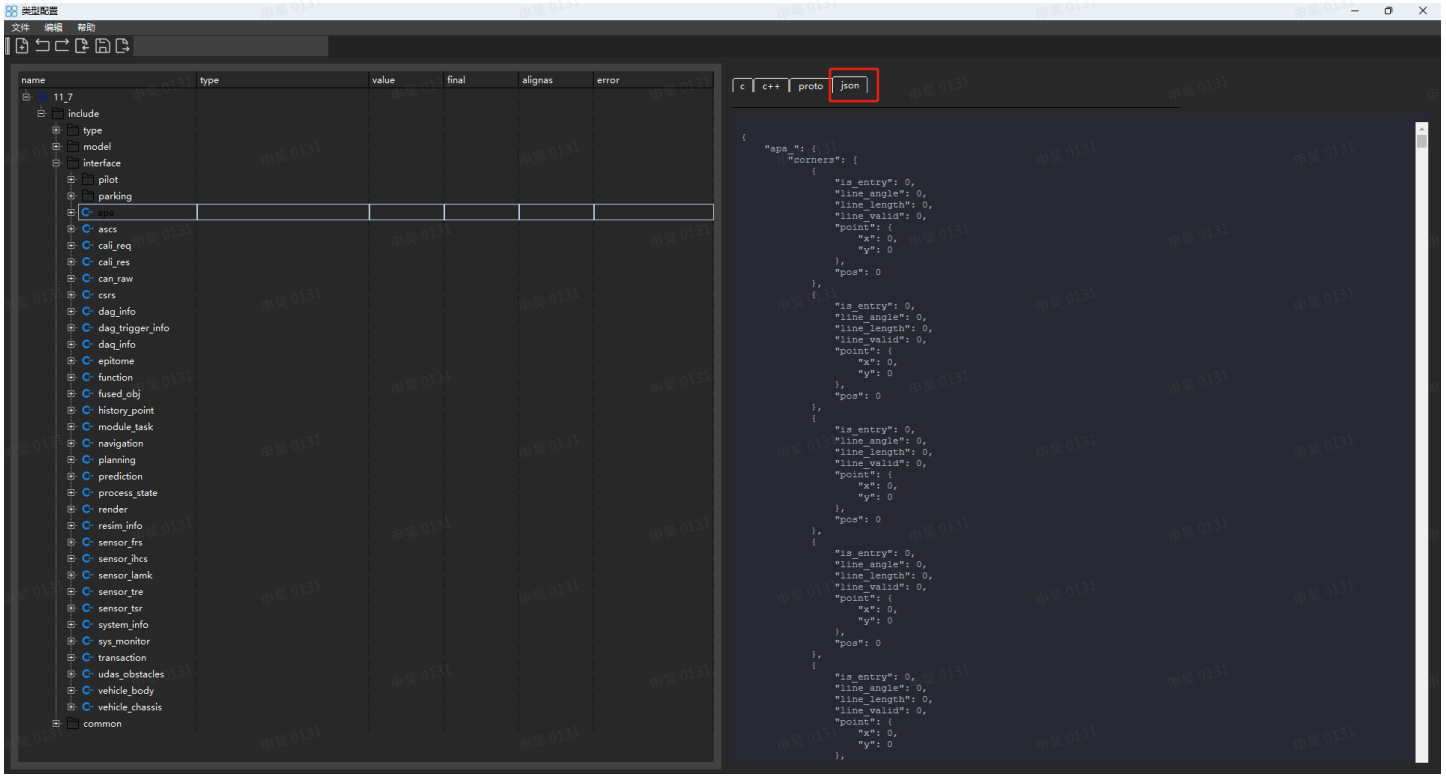

C风格头文件




PROTO文件




JSON文件



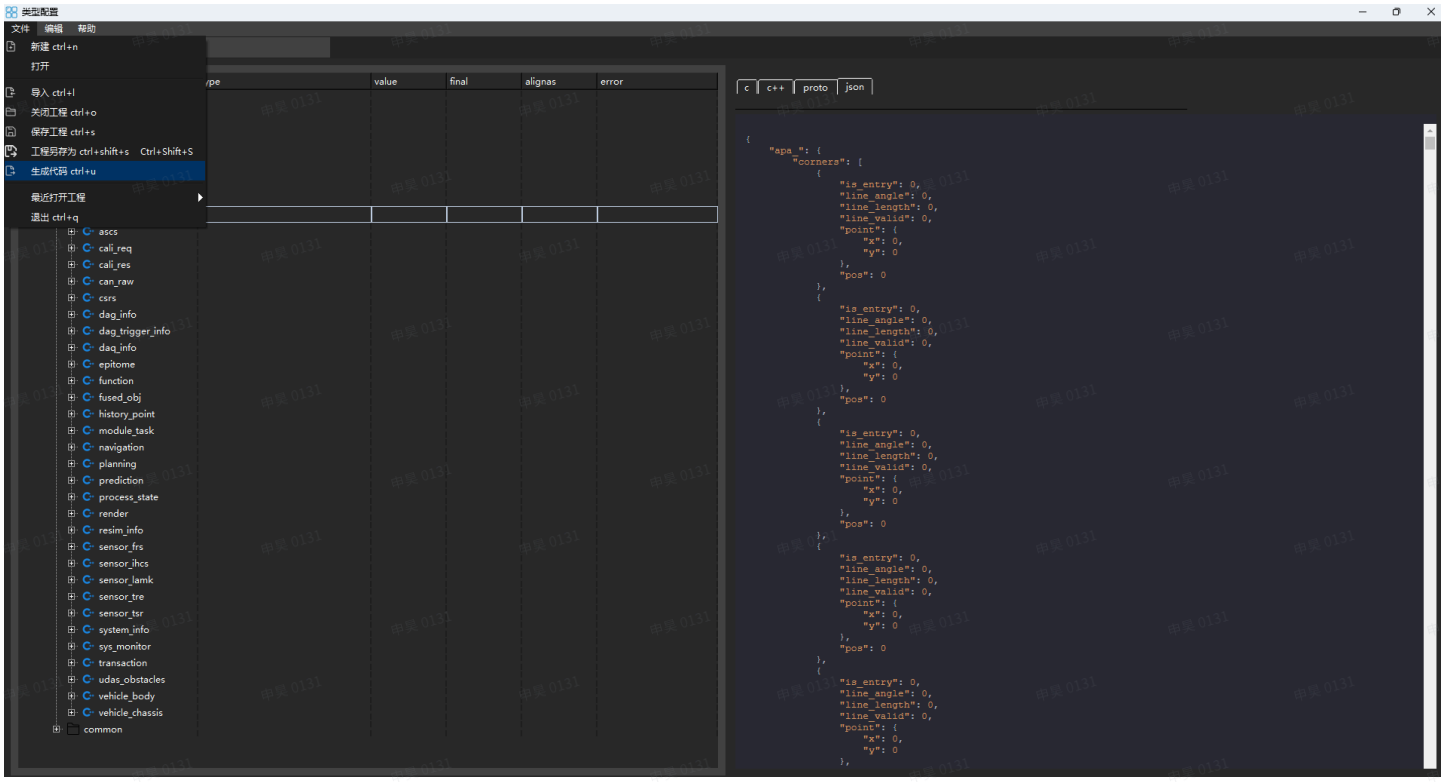

## 4.5 生成代码

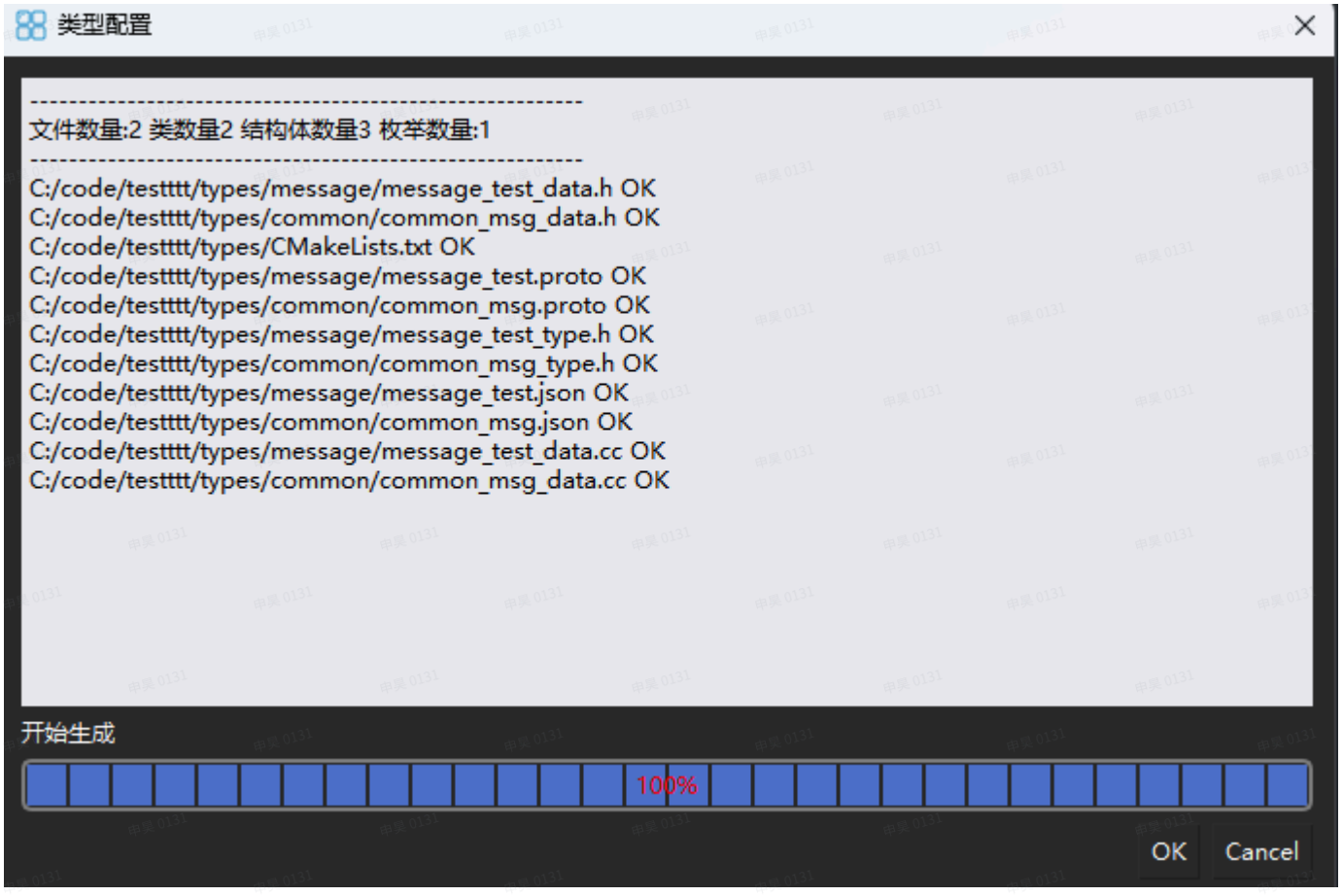
类型工具支持将类型区域的所有文件转换成多种风格的文件，分别是C风格的头文件(*_type.h)，c++风格的头文件(*_data.h)，proto文件(*.proto)，转换文件(*.cc)，protoc生成的pb文件(*.pb.h *.pb.cc)，json文件(*.json).

操作说明：

- 在菜单栏中点击“**文件**”。

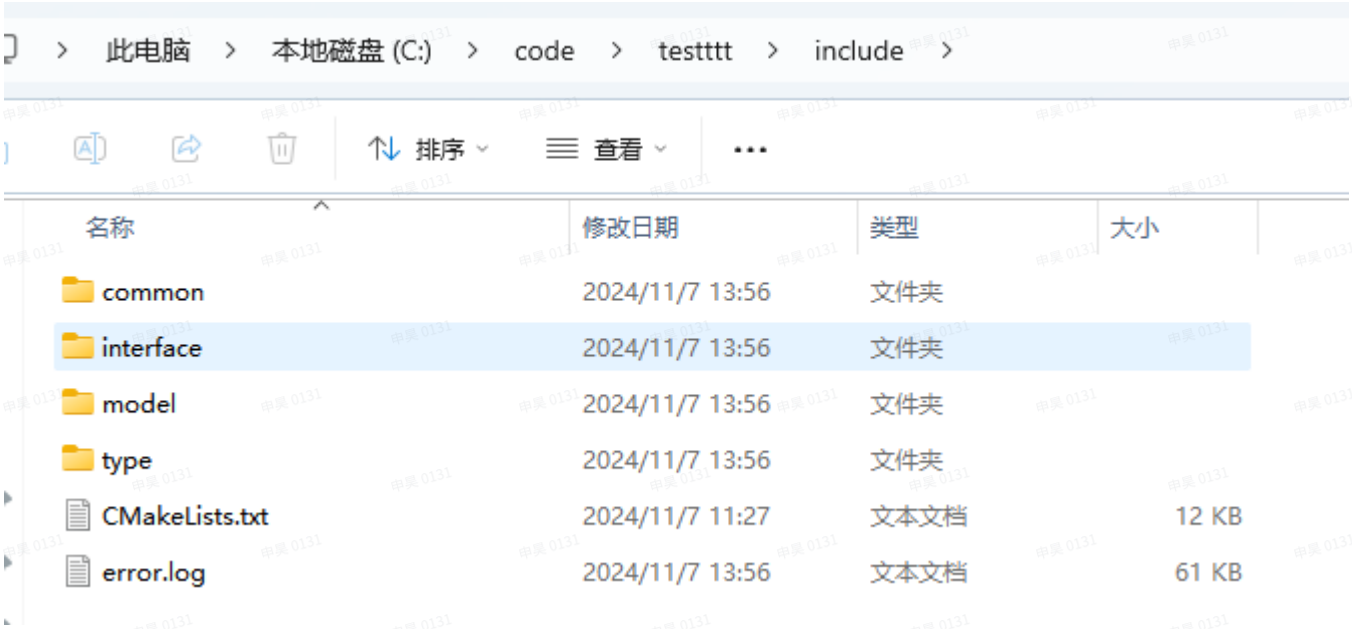
- 从下拉菜单中选择“**生成代码**”，或忽略以上第一条操作，直接点击菜单栏区的图标 
- 在弹出的对话框中，选择或输入希望生成的文件存放的路径。
- 确认选择无误后，点击“选择文件夹”按钮开始转换过程。
- 类型工具将根据选择在指定的路径下生成相应的文件。



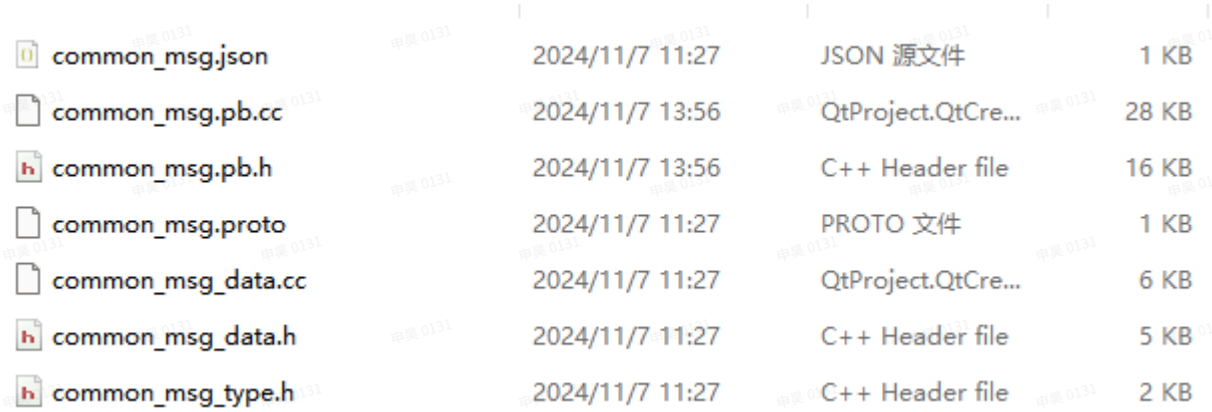



生成文件夹下会根据类型树的目录层级同步生成对应目录及其文件，除此以外，还会在工程目录下生成CMakeLists.txt，error.log(该文件仅在生成pb.cc，pb.h失败时创建）。

生成目录示例：




生成文件示例：




随后保存退出。

## 5. 快捷键介绍

该工具配备完整的快捷键操作

代码块

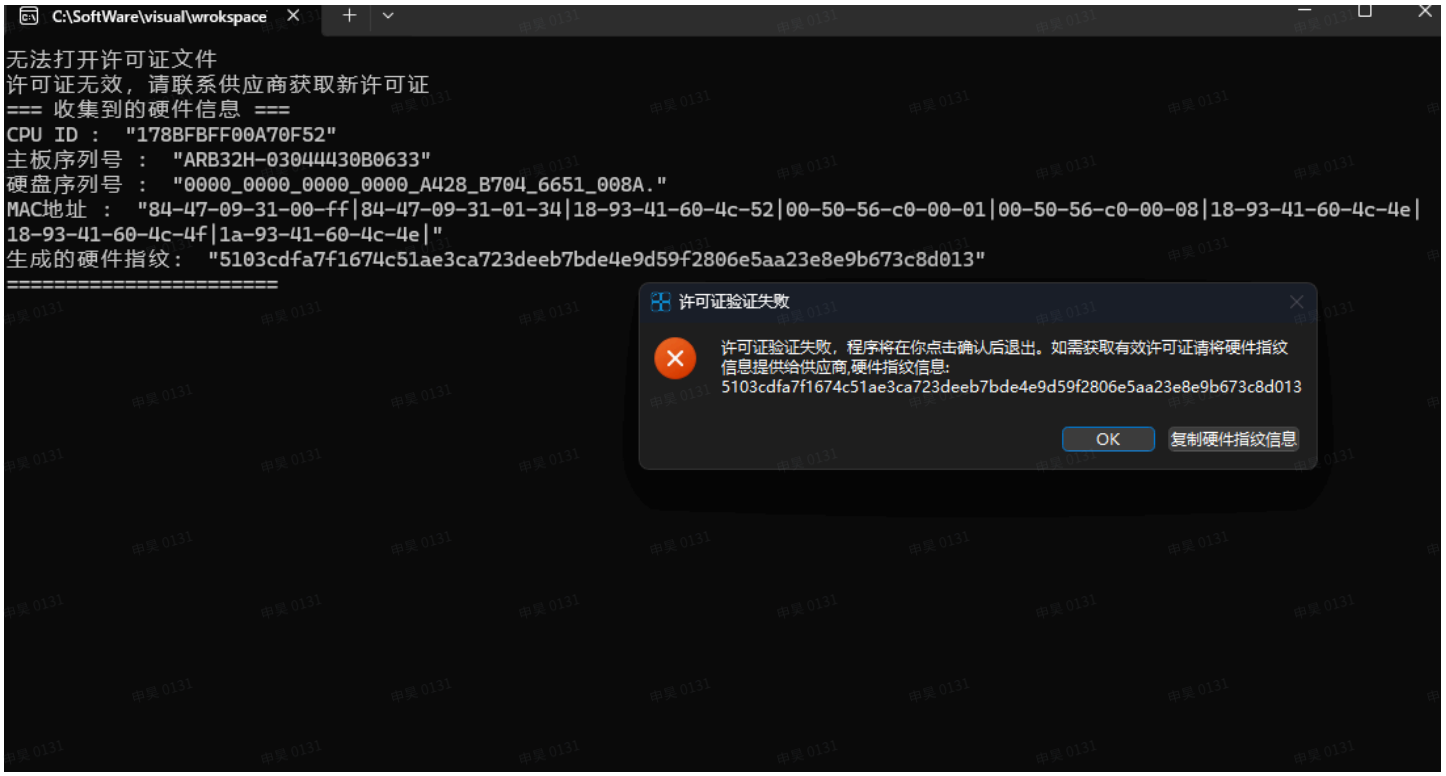
```
新建    CTRL + n
添加    CTRL + a
导入    CTRL + l
退出    CTRL + q
删除    CTRL + d
撤销    CTRL + z
重做    CTRL + y
```

| | | |
|---|---|---|
| 8 | 剪切 | CTRL + x |
| 9 | 复制 | CTRL + c |
| 10 | 粘贴 | CTRL + v |
| 11 | 上移 | CTRL + ⬆ |
| 12 | 下移 | CTRL + ⬇ |
| 13 | 搜索 | CTRL + f |
| 14 | 展开折叠 | CTRL + e |
| 15 | 关闭工程 | CTRL + o |
| 16 | 保存工程 | CTRL + s |
| 17 | 生成代码 | CTRL + u |
| 18 | 错误检查 | CTRL + p |
| 19 | 另存为 | CTRL + SHIFT + s |

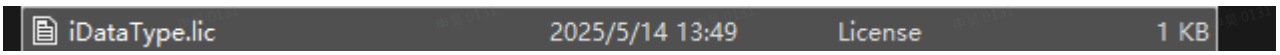
# 6.License证书验证

注意:使用iDataType前必须连接网络，不可以是局域网。

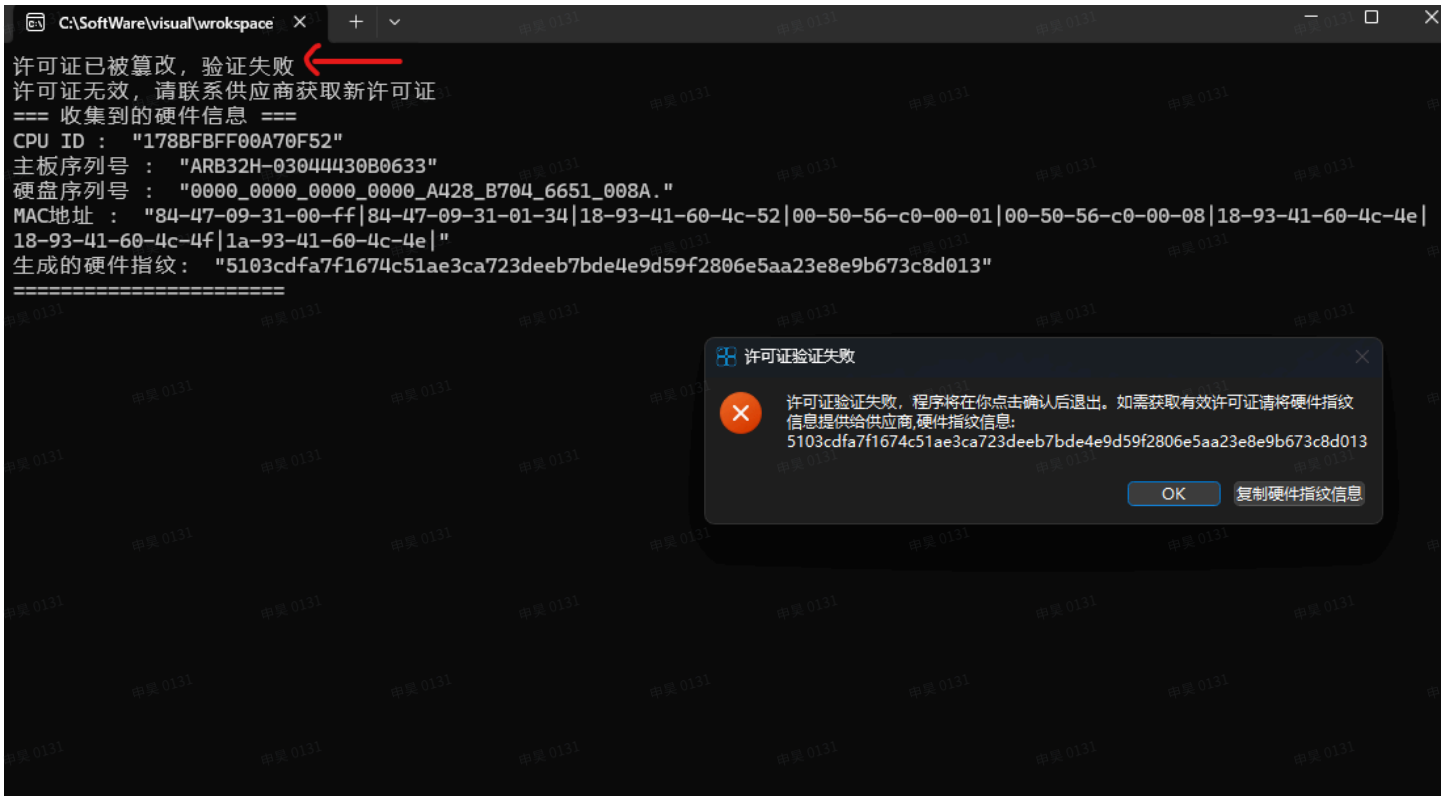
点击程序后如果没有有效的证书会如下提示，并自动获取硬件指纹信息，复制后提供给供应商



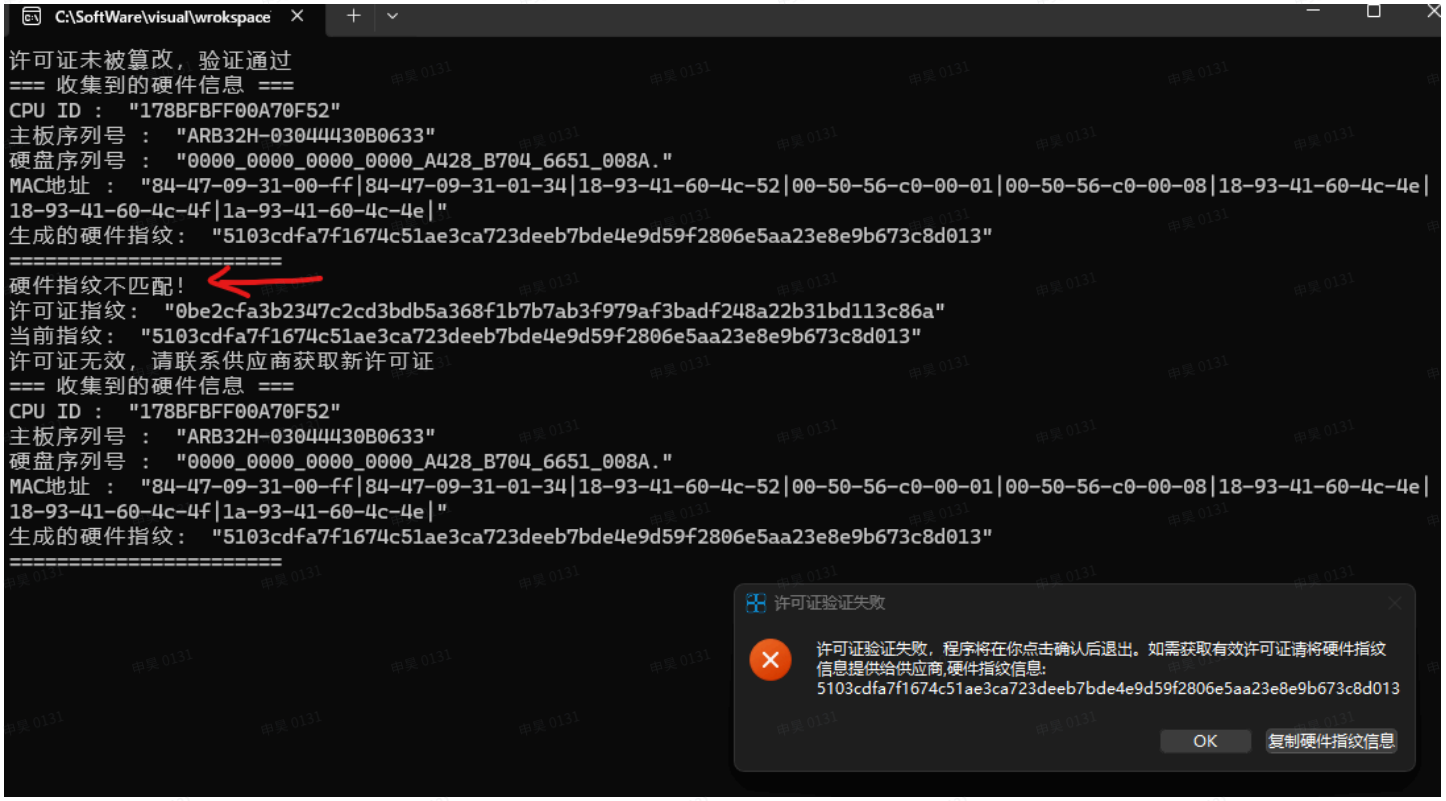

供应商会提供给license证书文件，iDataType工具的证书文件名称固定为iDataType.lic



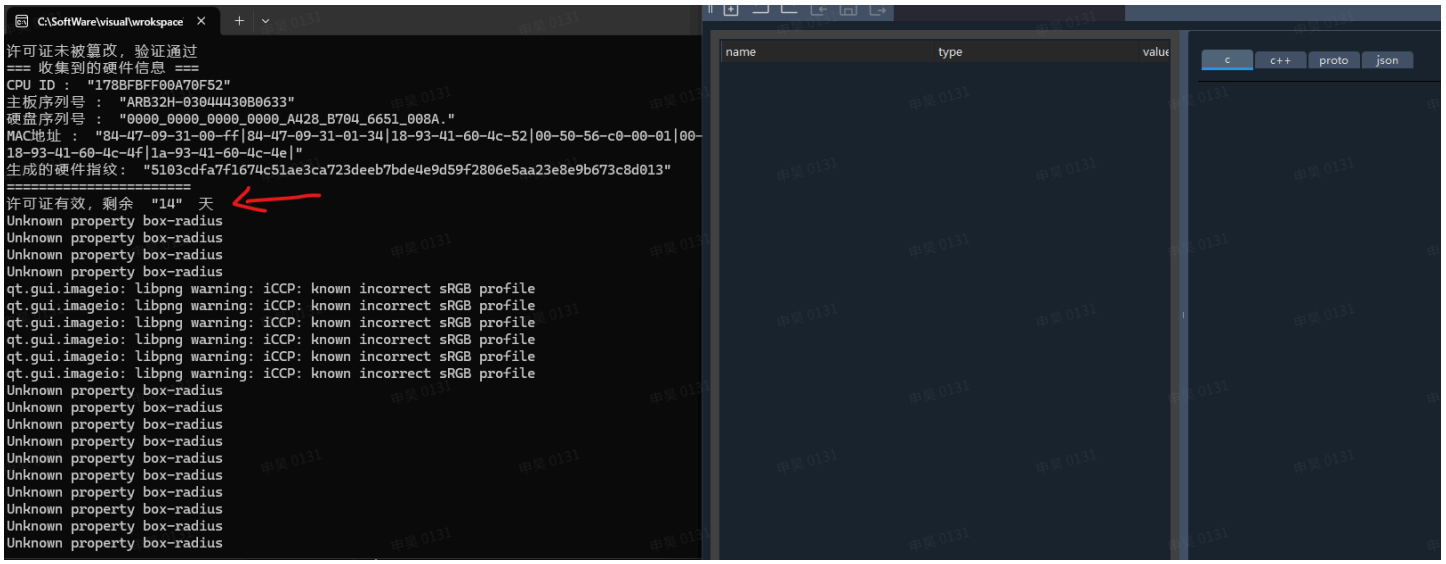

不可随意更改证书，否则会验证失败




如果硬件指纹不匹配也会验证失败




验证成功后即可使用iDataType


C:\SoftWare\visual\wrokspace
许可证未被篡改，验证通过
=== 收集到的硬件信息 ===
CPU_ID : "178BFBFF00A70F52"
主板序列号 : "ARB32H-030044430B0633"
硬盘序列号 : "0000_0000_0000_0000_A428_B704_6651_008A."
MAC地址 : "84-47-09-31-00-ff|84-47-09-31-01-34|18-93-41-60-4c-52|00-50-56-c0-00-01|00-18-93-41-60-4c-4f|1a-93-41-60-4c-4e|"
生成的硬件指纹: "5103cdfa7f1674c51ae3ca723deeb7bde4e9d59f2806e5aa23e8e9b673c8d013"
==========
许可证有效，剩余 "14" 天
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
qt.gui.imageio: libpng warning: iCCP: known incorrect sRGB profile
qt.gui.imageio: libpng warning: iCCP: known incorrect sRGB profile
qt.gui.imageio: libpng warning: iCCP: known incorrect sRGB profile
qt.gui.imageio: libpng warning: iCCP: known incorrect sRGB profile
qt.gui.imageio: libpng warning: iCCP: known incorrect sRGB profile
qt.gui.imageio: libpng warning: iCCP: known incorrect sRGB profile
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
Unknown property box-radius
name
type
value
c
c++
proto
json